Face

SQL: Structured Querry Language

It is a language used for interacting with data base.

What can SQL Do?

- SQL can execute queries on a database
- SQL can retrieve data from a database
- SQL can insert records in a database
- SQL can update records in a database
- SQL can delete records from a database
- SQL can create new tables in a database
- SQL can create views in a database

What is a Data base ?

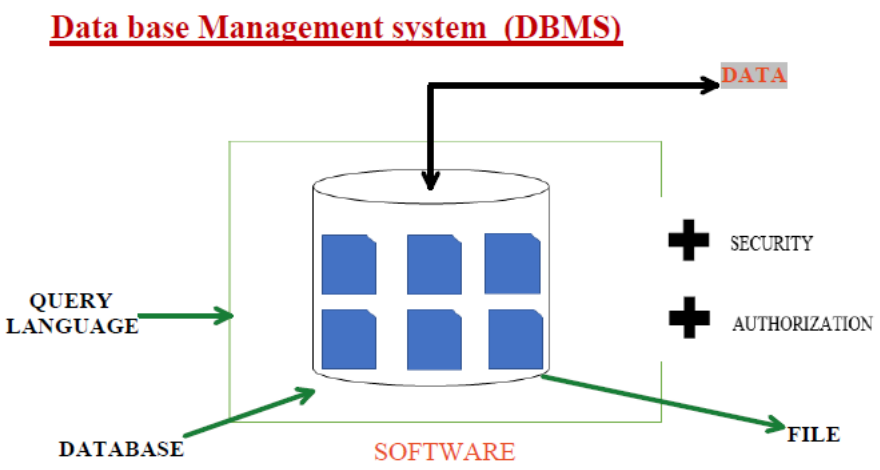
- It is a place where we are going to store all related information
- Data base is a place or media which can store the data in systematic and organized manner.

What does a Data base do ?

- The basic operations that can be performed on the database are:
  - Create/Insert
  - Read/ Retrieve
  - Update/ Modify
  - Drop/ Delete
- This operation are universally know as "CRUD" operation

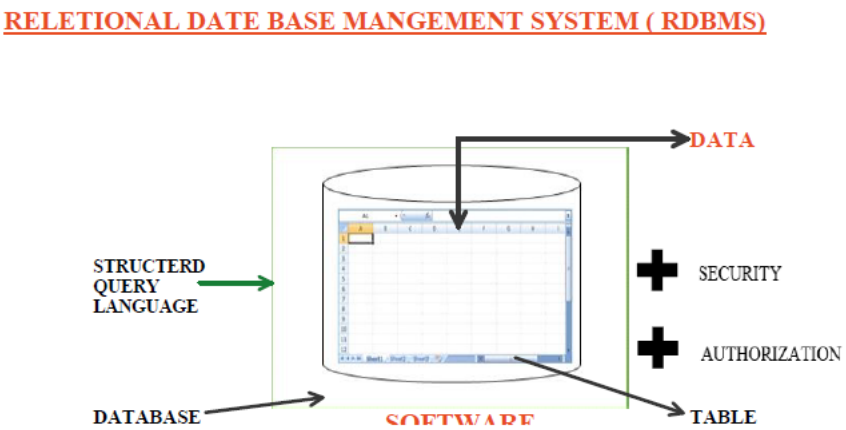
What is DBMS ?

- Data base management system is a software that will allow the users to create the data, maintain the data and share the data
- Ex: MS-Access, DB2 and Fox pro.
- Database is a software which is used to maintain and manage the database



- Security and Authorization are the two important key features provided by the DBMS.
- We use Query language to communicate or interact with the DBMS

- DBMS stores the data in the form of Files
  Relational Data Base Management System (RDBMS)



- Relational Data Base Management System is a collection of program that allows the users to create the data, maintain the data, share the data and Relate the data.
- Ex: Oracle SQL, Microsoft SQL Server, Sybase.
- RDBMS stores the data in the form of tables
- We can interact with RDBMS using SQL.

Interview Question:

When do we call any DBMS as RDBMS?

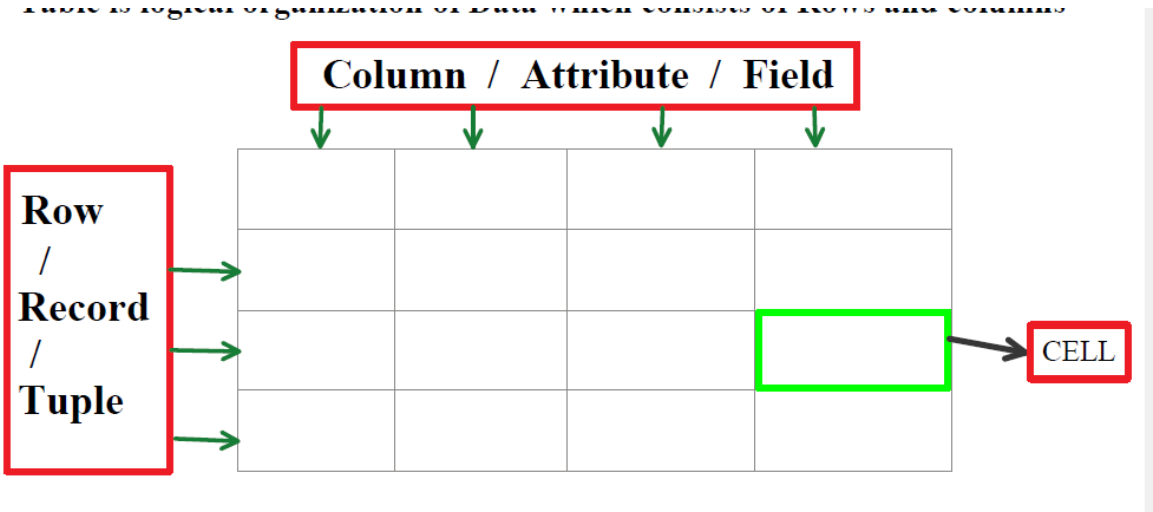
- If any DBMS supports minimum 6 CODD rules than these DBMS will be called RDBMS.

Relational Model

- Relational Model was designed by E.F CODD
- In relational model we store the data in form of Rows and Columns
- Any DBMS which follows Relational Model becomes on RDBMS

Table:

- Table is a logical organization of Data which consists of Rows and Columns



Columns:

- A column is also know as attributes or field
- A column is used to represent property of all the entity

Row:

- Row is also known as Record or Tuples
- Row is used to represent all properties of a single entities

Cell:

- Cell is the smallest unit of the table in which we store the data
- The interaction of rows and columns generate cells.

Example :

EMP :

EID	ENAME	SALARY
1	SMITH	1000
2	ALLEN	1500
3	CLARK	2000

Emp (Entity)
- Eid
- Ename
- Salary

Data entered into the table must be validated

They can be classified as two ways:

1. By assigning "Data Type"
2. By assigning "Constraints"

Data Type are mandatory where as **constrains** are optional

Data Type

Data Type is used to specify/ determine the type/kind of data that will be stored in a particular memory allocation.

Date Type in SQL

- **Char**
- **Varchar /Varchar2**
- **Number**
- **Date**
- **Large Object**
    - **Character Large Object (Clob)**
    - **Binary large Object(Blob)**
- **Note: SQL is not case sensitive language**

1. **Char:**

   In char data type we can store characters such as

   Example: '**A-Z', 'a-z', '0-9' and special characters ('$', '#', '%', '&')**

   When ever we use char data type we must specify the size

   Syntax: Char(size)

   Size-> Number of characters

   It is used to specify number of characters that can be stored

   We can store max of 2000 characters

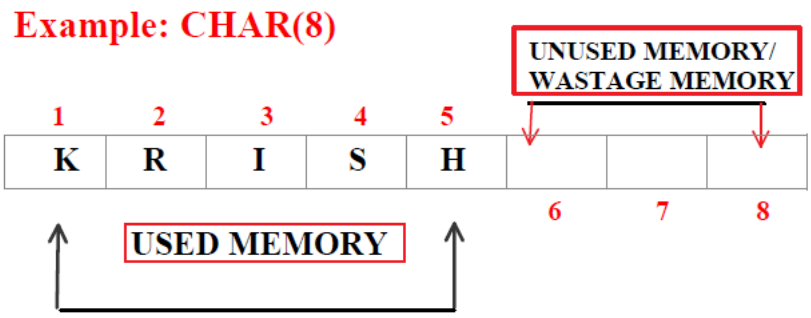
   Character must always be with in single quote (' ')

   Ex: 'ABC', 'RAJ', 'A'

   Character data type follows "fixed length memory allocation"

   Default size of char is one

   Char(8)



2. VARCHAR/VARCHAR2

In VARCHAR data type we can store characters such as

Example: ‘A-Z’, ‘a-z’, ‘0-9’ and special characters (‘$’ , ‘#’, ‘%’, ‘&’)

Whenever we use varchar data type we must specify the size

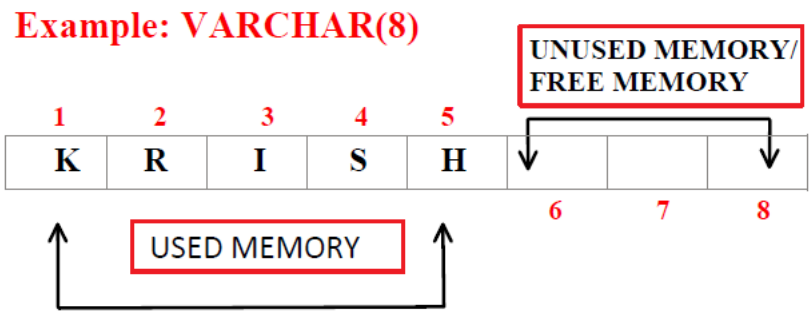
Syntax: varchar(size)

Size:
It is used to specify number of characters that can be stored

We can store a max of 2000 characters

Character must always be with in single quote (‘ ‘)
Ex: ‘ABC’, ‘Raj’, ‘A’

Varchar datatype follows variable length memory allocation default size of varchar is zero.



Note:

Varchar2 is nothing but update version of varchar

Max size is 4000 character

Assignment

1. Difference between char and varchar/varchar 2

3.Number

'This number data type is used to store numeric value'

Number data type can accept 2 arguments such as

Precision

Scale

Syntax: Number(precision [scale])

Precision

- Precision is used to determine the number of digit used to store 'Integer value'
- The range of the precision is 1 to 38

Scale:

- Scale is used to determine the number of digit used to store Decimal (Float) value within the precision
- The value of scale is 'Zero'
- The range of scale is -84 to 127.

Examples :

1	NUMBER(3)	+/-999
2	NUMBER(4)	+/-9999
3	NUMBER(5,2)	+/-999.99

PRECISION SCALE

5	NUMBER(7,3)	+/- 9999.999
6	NUMBER(3,6)	+/- 0.000999

4.Date

The Data datatype is used to store the date in specific format

Syntax: Date

Format:

1) 'DD-MON-YY' '28-JUN-21'

OR

2) 'DD-MON-YYYY' '28-JUN-2021'

5. Large Object

Character Large Object: (CLOB)

Character large Object (CLOB) is used to store huge amount of character up to 4GB of size.

SYNTAX

```
{CLOB | CHARACTER LARGE OBJECT } [(LENGTH [{K|M|G}])]
```

BINARY LARGE Object (BLOB)

Binary large object(BLOB) is used to store the binary values of “IMAGES, MP3, MP4, PDF” etc up-to 4GB of size.

SYNTAX

```
{BLOB | BINARY LARGE OBJECT } [(LENGTH [{K|M|G}])]
```

What are Constraints?

- **Definition:**
  A constraint in SQL is a rule applied to a column or table that ensures the validity, accuracy, and integrity of data.
- **Why needed?**
  Without constraints, the database would accept anything → duplicates, nulls, invalid values.
  Constraints make sure only meaningful and consistent data gets stored.

Think of constraints as "guards" at the entry gate of a table.

Types of Constraints in SQL Server

1. NOT NULL

- Ensures a column **cannot store NULL (empty) values**.
- Use when the field is **mandatory**.
- StudentID and Name cannot be NULL.
- Age can be NULL.

2. UNIQUE

- Ensures all values in a column are **unique** (no duplicates).
- Allows **NULL once** (in SQL Server).
- No two employees can have the same EmpID or Email.

3. CHECK

- Adds a **custom condition** to validate values.
- If the condition is **true**, the value is accepted; otherwise rejected.
- `Percentage` must be between 0–100.
- `Phone` must be exactly 10 digits.

4. PRIMARY KEY (PK)

- A column (or set of columns) that **uniquely identifies each record** in the table.
- Rules:
  - Only **one Primary Key per table**.
  - PK = **UNIQUE + NOT NULL** automatically.

Characteristics of Primary key :

- We can have only 1 PK in a table
- PK cannot accept duplicate / repeated values .

- PK cannot accept Null
- PK is always a combination of Unique and Not Null Constraint.
- PK is not mandatory but highly recommended.

Example: StudentID uniquely identifies each student.

5. FOREIGN KEY (FK)

- "*It is used to establish a connection between the The tables* "
- A column that **references a Primary Key in another table**.
- Used to **link two tables** (Parent–Child relationship).
- Enforces **Referential Integrity** → cannot insert a value in FK column unless it exists in parent PK.

Characteristics of Foreign key :

- We can have Multiple FK in a table
- FK can accept duplicate / repeated values .
- FK can accept Null
- FK is not a combination of Unique and Not Null Constraint.
- For an Attribute (column) to become a FK ,it is mandatory
- that it must be a PK in its own table .
- FK are present in child table but it actually belongs to parent table
- FK is also known as "REFRENTIAL INTEGRITY CONSTRAINT"

EMP

PK				
		CHECK(SALARY> 0)		***CHECK(LENTH(PHONE)= 10)***
NOT NULL UNIQUE	***NOT NULL***	***NOT NULL***	***NOT NULL***	***NOT NULL UNIQUE***
EID	***NAME***	***SALARY***	***DOJ***	***PHONE***
NUMBER(4)	***VARCHAR(10)***	***NUMBER(6,2)***	***DATE***	***NUMBER(10)***
123	***RAJU***	***5000.00***	***'26-JUN-1998'***	***1234567890***
321	***PRADEEP***	***6000.00***	***'28-JAN-2001'***	***3214567890***

Example for Foreign Key :

EMP

EID	NAME	SALARY	Cid(fk)	Dno(fk)
1	A	10000	1	10
2	B	20000	1	20
3	C	35000		20
4	D	50000	2	10

Child / accepter

CUSTOMER

CID	CNAME	CNO
1	X	1001
2	Y	2002
3	Z	3003

Parent/donar

DEPT

DNO	DNAME	LOC
10	D1	L1
20	D2	L2

Differentiate between Primary key and Foreign key .

Primary Key (PK)

- Used to **uniquely identify a record** in a table.
- Cannot accept **NULL** values.
- Cannot accept **duplicate** values.
- Always a combination of **NOT NULL** and **UNIQUE** constraint.
- Only **one Primary Key** is allowed per table.

Foreign Key (FK)

- Used to **establish a relationship/connection between two tables**.
- Can accept **NULL** values.
- Can accept **duplicate** values.
- Not a combination of NOT NULL and UNIQUE.
- A table can have **multiple Foreign Keys**.

NOTE : NULL

Null Is a keyword which is used to represent Nothing / Empty Cell.

Characteristics of Null :

- Null doesn't represent 0 or Space .
- Any operations performed on a Null will result in Null itself
- Null doesn't Occupy any Memory .
- We cannot Equate Null .

Ex:

emp

eid	ename	sal	comm
1	a	100	30
2	b	200	

200*null= null
200+null= null
200-null= null
200+ null= null

200=null (not possible)

SQL Statements

What is Statement in SQL ?

- A Statement in SQL is a command or instruction written in SQL language that tells the database what action to perform
- Actions can include creating tables, inserting data, updating records, retrieving information or controlling access.
- Each statement usually ends with a semicolon(;) not always required in SQL server but good practice.
- Think of SQL statements as the sentences you speak to the database in it own language.

Why do we need SQL Statements ?

- SQL statements are used to:
  - **Define** the structure of data(Tables, schemas, indexes)
  - **Manipulate** data (insert, update, delete)
  - **Retrieve** data (queries)
  - **Control** user access and permissions
  - **Maintain** transactions (commit, rollback)
- **Without statements, the DBMS wont know what action you want.**

Types of SQL statements (5 Categories)

SQL statements are broadly divided into 5 categories each serving a different purpose.

- **DDL**: CREATE, ALTER, TRUNCATE, DROP
- **DML**: INSERT, UPDATE, DELETE
- **DQL**: SELECT ... FROM ... WHERE ... ORDER BY ...
- **TCL**: BEGIN TRAN, SAVE TRAN, COMMIT, ROLLBACK
- **DCL**: GRANT, REVOKE

1. **DDL – Data Definition Language.**

- **Used** to define or modify the structure of the database(schema, tables, columns, constraints).
- **Key statements**
    - **CREATE ->** Create database objects (table, database, index, view)
    - **Alter ->** modify an existing object (add/remove column, change datatype)
    - **Drop ->** permanently delete a database object
    - **Truncate ->** remove all rows from a table but keep the structure
    - **Rename ->** Change the name of a database object

Creating a Database

Syntax

```
CREATE DATABASE DatabaseName;
```

Example

```
CREATE DATABASE SchoolDB;
```

This will create a new database named SchoolDB.

Using a Database

After creating, you must switch to that database before creating tables.

Syntax

USE DatabaseName;

Example

USE SchoolDB;

Creating a Table

Syntax:

```
CREATE TABLE TableName (
    ColumnName1 DataType Constraints,
    ColumnName2 DataType Constraints,
    ...
);
```

TableName → Name of your table.

ColumnName → Attribute/field name.

DataType → INT, VARCHAR, DATE, etc.

Constraints → Rules (PRIMARY KEY, NOT NULL, UNIQUE, DEFAULT, CHECK).

Example: Create Students Table

```
CREATE TABLE Students (
   StudentID INT PRIMARY KEY,     -- unique student identifier
   Name VARCHAR(50) NOT NULL,     -- cannot be empty
   Age INT CHECK (Age > 0),       -- must be positive
   Marks INT DEFAULT 0            -- default value = 0 if not provided
);
```

Inserting Records into the Table

Syntax

```
INSERT INTO TableName (col1, col2, col3, ...)
VALUES (value1, value2, value3, ...);
```

Example:

```
INSERT INTO Students (StudentID, Name, Age, Marks)
VALUES (101, 'Rahul', 16, 85);

INSERT INTO Students (StudentID, Name, Age)
VALUES (102, 'Priya', 17);   -- Marks will default to 0
```

Retrieving Data (First SELECT)

```
SELECT * FROM Students;
```

Output might look like:

StudentID	Name	Age	Marks
101	Rahul	16	85
102	Priya	17	0

Deleting a Table or Database

Delete Table

DROP TABLE Students;

Delete Database

DROP DATABASE SchoolDB;

Note: You must USE master; before dropping a DB, and disconnect active sessions.

USE master;

ALTER DATABASE SchoolDB SET SINGLE_USER WITH ROLLBACK IMMEDIATE;

DROP DATABASE SchoolDB;

DDL Statements Continuation

ALTER

Definition: Used to change table structure (add/remove columns, change datatype).

Syntax:
ALTER TABLE TableName ADD ColumnName DataType;

ALTER TABLE TableName ALTER COLUMN ColumnName NewDataType;

ALTER TABLE TableName DROP COLUMN ColumnName;

Query:

ALTER TABLE Students ADD Email VARCHAR(100);

ALTER TABLE Students ALTER COLUMN Name VARCHAR(80); -- increase size

ALTER TABLE Students DROP COLUMN Email;

TRUNCATE

Definition: Removes all data from a table but keeps its structure. Faster than DELETE.

Syntax:
TRUNCATE TABLE TableName;

Example:

TRUNCATE TABLE Students;

Note: After truncating, Students table is empty but still exists.

DROP

Definition: Permanently deletes the object (table or database).

Syntax
DROP TABLE TableName;

DROP DATABASE DatabaseName;

Example:
DROP TABLE Students;

USE master;

ALTER DATABASE SchoolDB SET SINGLE_USER WITH ROLLBACK IMMEDIATE;

DROP DATABASE SchoolDB;

Warning: Cannot be rolled back once executed.

2. DML (Data Manipulation Language)

Definition

- DML statements are used to **insert, update, and delete data stored inside tables**.
- They don't change the structure of the database, only the **content**.

Key DML Statements

1. INSERT → Add new records.
2. UPDATE → Modify existing records.
3. DELETE → Remove records.

INSERT

Syntax:

INSERT INTO TableName (col1, col2, col3, ...)

VALUES (val1, val2, val3, ...);

Example:
INSERT INTO Students (StudentID, Name, Age, Marks)

VALUES (101, 'Rahul', 16, 85);

INSERT INTO Students (StudentID, Name, Age)

VALUES (102, 'Priya', 17); -- Marks will default to 0

In **SQL Server**, you can insert **multiple rows** in a single INSERT query by separating each record with a comma.

Syntax:

```
INSERT INTO TableName (col1, col2, col3, ...)
VALUES
  (value1a, value2a, value3a, ...),
  (value1b, value2b, value3b, ...),
  (value1c, value2c, value3c, ...);
```

Example with your Students table:

```
INSERT INTO Students (StudentID, Name, Age, Marks)
VALUES
  (105, 'Amit', 17, 88),
  (106, 'Sara', 16, 92),
  (107, 'Kiran', 18, 75);
```

UPDATE

Syntax:

```
UPDATE TableName
SET col1 = val1, col2 = val2
WHERE condition;
```

Example:

```
UPDATE Students
SET Marks = 90
WHERE StudentID = 101;
```

DELETE

Syntax:

```
DELETE FROM TableName WHERE condition;
```

Example:

DELETE FROM Students

WHERE StudentID = 102;

Without WHERE, all rows will be deleted:

Example

DELETE FROM Students;

3. TCL (Transaction Control Language)

Definition

- TCL statements are used to **manage transactions** — groups of SQL commands that are executed as a single unit.
- They ensure **data integrity** using the ACID properties.

What are Transactions?

- A transaction is like a package of SQL commands that must all succeed together or fail together.
- **Example**: In banking, when you transfer money, one account must be debited and another credited. Both must succeed, or both must fail — you can't let only one happen.

What does TCL do?

- TCL (Transaction Control Language) helps you **control these transactions**.
- Think of it as the **remote control** for database actions: you decide whether to keep them, undo them, or pause at a checkpoint.

Why is it needed? (ACID properties)
Transactions must follow **ACID** rules so that data always stays correct:

- **Atomicity** → All or nothing.
  (Money debit + credit both happen, or neither.)
- **Consistency** → Database always stays valid.
  (Balances remain correct before and after.)
- **Isolation** → Transactions don't disturb each other.
  (Two people withdrawing at the same time won't clash.)
- **Durability** → Once saved, it's permanent.
  (Power cut won't lose committed changes.)

BEGIN TRAN → Start transaction.

SAVE TRAN → Create checkpoint.

COMMIT → Save permanently.

ROLLBACK → Undo changes.

Key TCL Statements

1. COMMIT → Save all changes permanently.
2. ROLLBACK → Undo changes since last commit.
3. SAVEPOINT → Create a checkpoint inside a transaction.

COMMIT → Save the changes permanently.

ROLLBACK → Undo changes before saving.

SAVEPOINT → Mark a checkpoint so you can roll back partially.

COMMIT

Syntax: COMMIT;

Example:

```
BEGIN TRAN;

UPDATE Students SET Marks = 95 WHERE StudentID = 101;

COMMIT;  -- changes saved
```

ROLLBACK

Syntax:

```
ROLLBACK;
```

Example:

```
BEGIN TRAN;

DELETE FROM Students WHERE StudentID = 101;

ROLLBACK;  -- undo delete
```

SAVEPOINT

Syntax:

SAVE TRAN SavepointName;

ROLLBACK TRAN SavepointName;

Example:

```
BEGIN TRAN;

INSERT INTO Students (StudentID, Name, Age, Marks)
VALUES (103, 'John', 15, 70);

SAVE TRAN sp1;

INSERT INTO Students (StudentID, Name, Age, Marks)
VALUES (104, 'Anita', 18, 80);

-- rollback only second insert
ROLLBACK TRAN sp1;

COMMIT;  -- first insert saved, second undone
```

4. DQL (Data Query Language)

Definition

- DQL statements are used to **retrieve and view data** from the database.
- They do not modify the structure or the content of data, only query it.
- In SQL Server, **SELECT** is the main DQL command.

Why is DQL important?

- Without DQL, databases would just store data but you couldn't analyze or use it.
- DQL makes data meaningful by turning it into **information** (e.g., fetching only top students, or filtering sales for last month).

SELECT

SYNTAX:

```
SELECT column1, column2, ...
FROM TableName
WHERE condition
ORDER BY column ASC|DESC;
```

Example 1: Retrieve all columns

```
SELECT * FROM Students;
```

Example 2: Retrieve selected columns

```
SELECT Name, Marks FROM Students;
```

Example 3: Filter records

```
SELECT * FROM Students
WHERE Marks > 80;
```

Example 4: Sort records

```
SELECT * FROM Students
ORDER BY Marks DESC;
```

5. DCL (Data Control Language)

Definition

- DCL statements are used to **control access and permissions** in the database.
- They ensure **security** by deciding *who can do what* in the system.

Key DCL Statements

1. GRANT → Give user permission.
2. REVOKE → Remove user permission .

GRANT

Syntax:

```
GRANT permission_type
ON object_name
```

```
TO user_name;
```

Example

GRANT SELECT, INSERT

ON Students

TO John;

This allows user *John* to **read** and **insert** into the Students table.

REVOKE

Syntax:

```
REVOKE permission_type
ON object_name
FROM user_name;
```

Example

```
REVOKE INSERT
ON Students
FROM John;
```

Now John can only read (SELECT) but cannot insert anymore.

Interview Questions:

DBMS (Database Management System)

- **Definition**:
  A DBMS is software used to store and manage data. Data in DBMS is often stored in **files** rather than in strictly organized relational tables.
- **Key Points**:
  - DBMS does **not support client–server architecture** → it's usually used on a single system.
  - DBMS generally does **not enforce integrity constraints**.
    - **Integrity constraints** = rules to ensure correctness and consistency of data.
    - Example: Preventing duplicate student roll numbers, ensuring an age cannot be negative.

    - DBMS was first introduced in the **1960s by Charles Bachman**.
    - Data is often organized as an **inverted tree** (hierarchical model) → one parent, many children.
    - Supports **single user only**, best suited for small-scale applications.
- **Examples**:
  Microsoft Access, dBASE, early file-based systems.

RDBMS (Relational Database Management System)

- **Definition**:
  An RDBMS is an advanced DBMS that stores data in **tables (rows and columns)** and enforces **relationships between tables**.
- **Key Points**:
    - Data is stored in **two-dimensional tables** (called **relations**).
    - Supports **client–server architecture** → multiple users can access it simultaneously.
    - Imposes **integrity constraints** to maintain accuracy. Common constraints:
        - **Primary Key** → uniquely identifies each record.
        - **Foreign Key** → links two tables.
        - **Unique, Not Null, Check** constraints.
    - Follows **Normalization**:
        - Process of organizing data into tables to reduce duplication and improve consistency.
        - Example: Instead of repeating department name for every student, create a Departments table and reference it using a foreign key.
    - RDBMS was introduced by **Dr. E.F. Codd (IBM, 1970s)**.
    - Designed for **multi-user, large-scale operations**.
- **Examples**:
  Microsoft SQL Server, Oracle, MySQL, PostgreSQL.

Difference between:

- Delete & Truncate
- Delete & Drop
- Drop & Truncate
- DDL & DML
- Insert & Alter
- Alter & update
- Commit & RollBack

- Commit & Save point

1. DELETE vs TRUNCATE

- **DELETE**
    - DML command (Data Manipulation Language).
    - Removes rows **one by one** (row-level operation).
    - Can use WHERE clause → delete specific rows.
    - Slower for large data (logs each row).
    - Can be rolled back (inside a transaction).
    - Triggers **do fire**.
- **TRUNCATE**
    - DDL command (Data Definition Language).
    - Removes **all rows at once** (bulk operation).
    - Cannot use WHERE (clears full table).
    - Much faster for large data.
    - Cannot be rolled back (auto-commit).
    - Triggers **do not fire**.

Interview Tip: DELETE = row-by-row, TRUNCATE = bulk clear.

2. DELETE vs DROP

- **DELETE** → Removes only the data, table structure remains.
- **DROP** → Deletes the table **and its structure** completely.
- DELETE is reversible (with rollback), DROP is permanent.
- DELETE = content removal; DROP = object removal.

Interview Tip: DELETE affects rows, DROP affects the whole table.

3. DROP vs TRUNCATE

- **DROP**
    - Deletes the **entire table** (data + structure).
    - Cannot be rolled back.
    - Table must be recreated if needed again.
- **TRUNCATE**
    - Deletes only the **data**, keeps the structure.
    - Cannot be rolled back.

    - Can immediately re-insert new data.

Interview Tip: DROP = removes table, TRUNCATE = clears table.

4. DDL vs DML

- **DDL (Data Definition Language)**
    - Defines database structure (schema).
    - Examples: CREATE, ALTER, DROP, TRUNCATE.
    - Auto-committed, no rollback.
- **DML (Data Manipulation Language)**
    - Manipulates data inside tables.
    - Examples: INSERT, UPDATE, DELETE.
    - Can be rolled back (transaction safe).

Interview Tip: DDL = structure, DML = data.

5. INSERT vs ALTER

- **INSERT**
    - DML command.
    - Adds new records (rows) to a table.
    - Deals with **data only**.
- **ALTER**
    - DDL command.
    - Changes table structure (add/drop/modify columns).
    - Deals with **schema only**.

Interview Tip: INSERT = add data, ALTER = change structure.

6. ALTER vs UPDATE

- **ALTER**
    - DDL command.
    - Modifies table design (columns, constraints).
    - Affects schema, not row data.
- **UPDATE**
    - DML command.
    - Changes values in existing rows.
    - Affects data, not schema.

Interview Tip: ALTER = schema change, UPDATE = data change.

7. COMMIT vs ROLLBACK

- **COMMIT**
  - Saves all changes permanently.
  - Cannot be undone once committed.
  - Makes changes visible to other users.
- **ROLLBACK**
  - Undoes changes since the last COMMIT.
  - Restores database to a safe state.
  - Used when an error/mistake occurs.

Interview Tip: COMMIT = save, ROLLBACK = undo.

8. COMMIT vs SAVEPOINT

- **COMMIT**
  - Saves all changes permanently.
  - No undo possible after commit.
  - Ends the transaction.
- **SAVEPOINT**
  - Creates a checkpoint inside a transaction.
  - Lets you roll back **part of a transaction** instead of the whole.
  - Does not end the transaction.

Interview Tip: COMMIT = full stop, SAVEPOINT = bookmark inside transaction.

1. Create Database

```
CREATE DATABASE CompanyDB;

USE CompanyDB;
```

2. Create Department Table (dept)

```
CREATE TABLE dept (
   DEPTNO INT PRIMARY KEY,
   DNAME VARCHAR(50),
   LOC VARCHAR(50)
);
```

3. Insert Records into dept

```
INSERT INTO dept (DEPTNO, DNAME, LOC) VALUES
(10, 'ACCOUNTING', 'NEW YORK'),
(20, 'RESEARCH', 'DALLAS'),
(30, 'SALES', 'CHICAGO'),
(40, 'OPERATIONS', 'BOSTON');
```

4. Create Employee Table (emp)

```
CREATE TABLE emp (
   EMPNO INT PRIMARY KEY,
   ENAME VARCHAR(50),
   JOB VARCHAR(50),
   MGR INT NULL,
   HIREDATE DATE,
   SAL INT,
   COMM INT NULL,
   DEPTNO INT FOREIGN KEY REFERENCES dept(DEPTNO)
);
```

5. Insert Records into emp

```
INSERT INTO emp (EMPNO, ENAME, JOB, MGR, HIREDATE, SAL, COMM, DEPTNO) VALUES
(7369, 'SMITH', 'CLERK', 7902, '1980-12-17', 800, NULL, 20),
(7499, 'ALLEN', 'SALESMAN', 7698, '1981-02-20', 1600, 300, 30),
(7521, 'WARD', 'SALESMAN', 7698, '1981-02-22', 1250, 500, 30),
(7566, 'JONES', 'MANAGER', 7839, '1981-04-02', 2975, NULL, 20),
(7654, 'MARTIN', 'SALESMAN', 7698, '1981-09-28', 1250, 1400, 30),
(7698, 'BLAKE', 'MANAGER', 7839, '1981-05-01', 2850, NULL, 30),
(7782, 'CLARK', 'MANAGER', 7839, '1981-06-09', 2450, NULL, 10),
(7788, 'SCOTT', 'ANALYST', 7566, '1987-07-13', 3000, NULL, 20),
(7839, 'KING', 'PRESIDENT', NULL, '1981-11-17', 5000, NULL, 10),
(7844, 'TURNER', 'SALESMAN', 7698, '1981-09-08', 1500, 0, 30),
(7876, 'ADAMS', 'CLERK', 7788, '1987-05-23', 1100, NULL, 20),
(7900, 'JAMES', 'CLERK', 7698, '1981-12-03', 950, NULL, 30),
(7902, 'FORD', 'ANALYST', 7566, '1981-12-03', 3000, NULL, 20),
(7934, 'MILLER', 'CLERK', 7782, '1982-01-23', 1300, NULL, 10);
```

What is an Operator in SQL?

- An **operator** in SQL is a **special symbol or keyword** used to perform an operation on values.
- Think of it as a **tool** that helps SQL **compare, calculate, or combine** data in queries.

In short: Operators are like the **+ , - , = , >** signs we use in math or logic, but inside SQL.

Why do we need Operators in SQL?

We need operators because:

1. **To filter data** → Example: Find all employees with salary > 2000 (> operator).
2. **To compare values** → Example: Check if department number = 20 (= operator).
3. **To combine conditions** → Example: Employees in dept 10 **AND** salary > 1000 (AND operator).
4. **To perform calculations** → Example: Salary + Commission (+ operator).
5. **To check special cases** → Example: Find rows where commission is **NULL** (IS NULL operator).

🔹 1. Arithmetic Operators

Used for mathematical operations on numeric data.

- + → Addition
- - → Subtraction
- * → Multiplication
- / → Division
- % → Modulus (remainder after division – supported in SQL Server/MySQL, not in all DBs)

🔹 2. Concatenation Operator

Used to join strings/text.

- || → Concatenate strings (Oracle, PostgreSQL)
- + → Concatenate strings (SQL Server)

🔹 3. Comparison Operators

Used to compare values, return **TRUE / FALSE / UNKNOWN**.

- = → Equal to

- <> or != → Not equal to
- > → Greater than
- < → Less than
- >= → Greater than or equal to
- <= → Less than or equal to

🔹 4. Logical Operators

Used to combine multiple conditions.

- AND → True if **all** conditions are true
- OR → True if **any one** condition is true
- NOT → Reverses the result (True → False, False → True)

🔹 5. Special Operators

Used for advanced filtering/matching.

- IN → Match against a list of values
- NOT IN → Exclude a list of values
- BETWEEN ... AND ... → Within a range (inclusive)
- NOT BETWEEN ... AND ... → Outside a range
- IS NULL → Value is NULL
- IS NOT NULL → Value is NOT NULL
- LIKE → Pattern matching (%, _)
- NOT LIKE → Opposite of LIKE

Arithmetic operators

```
-- +  Add: salary + commission (treat NULL commission as 0)
SELECT ENAME, SAL, COMM, SAL + ISNULL(COMM, 0) AS TotalPay
FROM emp;
```

Note: ISNULL(COMM, 0) is just telling SQL Server:

"if COMM is empty (NULL), treat it as 0."

```
-- -  Subtract: salary after a flat tax of 200
SELECT ENAME, SAL, SAL - 200 AS NetAfterFlatTax
FROM emp;

-- *  Multiply: annual salary (12 months)
SELECT ENAME, SAL, SAL * 12 AS AnnualSalary
FROM emp;

-- /  Divide: approximate per-day pay (30-day month)
SELECT ENAME, SAL, CAST(SAL / 30.0 AS DECIMAL(10,2)) AS PerDayPay
FROM emp;
```

Why CAST is used here?

- In SQL Server, if you divide integers, the result is also an integer (it cuts off decimals).
- Example: 100 / 30 = 3 (not 3.33).
- To fix this, we use CAST to convert it into a decimal so SQL gives us a **precise value** like 3.33.

🔹 2. Concatenation Operator (|| or + in SQL Server)

Used to **combine text (strings)**.

Example (SQL Server uses +):

```
SELECT ENAME + ' works as ' + JOB AS EmployeeDetails
FROM emp;
```

Output: "SMITH works as CLERK".

🔹 3. Comparison Operators (=, != or <>)

Used to compare values.

Example:

```
-- Employees in department 30
SELECT ENAME, DEPTNO
```

```
FROM emp
WHERE DEPTNO = 30;

-- Employees not in dept 30
SELECT ENAME, DEPTNO
FROM emp
WHERE DEPTNO <> 30;
```

- **4. Relational Operators (>, <, >=, <=)**

Used to compare greater than, less than, etc.

Example:

```
-- Employees earning more than 2000
SELECT ENAME, SAL
FROM emp
WHERE SAL > 2000;

-- Employees earning less than or equal to 1000
SELECT ENAME, SAL
FROM emp
WHERE SAL <= 1000;
```

- **5. Logical Operators (AND, OR, NOT)**

Used to **combine conditions**.

Examples:

```
-- Employees in dept 30 earning more than 1500
SELECT ENAME, DEPTNO, SAL
FROM emp
WHERE DEPTNO = 30 AND SAL > 1500;

-- Employees in dept 10 OR dept 20
SELECT ENAME, DEPTNO
```

```
FROM emp
WHERE DEPTNO = 10 OR DEPTNO = 20;
```

-- Employees NOT in dept 10

```
SELECT ENAME, DEPTNO
FROM emp
WHERE NOT DEPTNO = 10;
```

🔹 6. Special Operators

a) IN

Checks if a value is in a list.

```
SELECT ENAME, DEPTNO
FROM emp
WHERE DEPTNO IN (10, 20);
```

b) NOT IN

Opposite of IN.

```
SELECT ENAME, DEPTNO
FROM emp
WHERE DEPTNO NOT IN (10, 20);
```

c) BETWEEN

Checks if a value is **between two values** (inclusive).

```
SELECT ENAME, SAL
FROM emp
WHERE SAL BETWEEN 1000 AND 2000;
```

d) NOT BETWEEN

Opposite of BETWEEN.

```
SELECT ENAME, SAL
FROM emp
WHERE SAL NOT BETWEEN 1000 AND 2000;
```

e) IS

Used to check for **NULL values**.

```
SELECT ENAME, COMM
FROM emp
WHERE COMM IS NULL;
```

f) IS NOT

Opposite of IS.

```
SELECT ENAME, COMM
FROM emp
WHERE COMM IS NOT NULL;
```

g) LIKE

Used for **pattern matching** (wildcards).

- % → any number of characters.
- _ → single character.

```
-- Names starting with 'S'
SELECT ENAME
FROM emp
WHERE ENAME LIKE 'S%';

-- Names with 5 letters only
SELECT ENAME
FROM emp
WHERE ENAME LIKE '_____';
```

h) NOT LIKE

Opposite of LIKE.

```
-- Employees whose name does NOT start with 'S'
SELECT ENAME
FROM emp
WHERE ENAME NOT LIKE 'S%';
```

Interview Questions

1. What are operators in SQL and why are they needed?

Answer:

- Operators are symbols or keywords that perform operations on values.
- They help filter, compare, calculate, or combine data inside queries.
- Example: WHERE SAL > 2000 → > is an operator checking salary greater than 2000.

2. Difference between comparison operators (=, <>, >, <) and logical operators (AND, OR, NOT)?

Answer:

- Comparison operators compare one column/value with another and return TRUE/FALSE.
- Logical operators combine multiple conditions.
- Example:

  SELECT ENAME FROM emp

  WHERE DEPTNO = 10 AND SAL > 2000;

  Here = and > are comparison, AND is logical.

3. How does BETWEEN differ from using >= and <=?

Answer:

- BETWEEN is shorthand for checking inclusive range.
- Example:

  WHERE SAL BETWEEN 1000 AND 2000

  is same as

  WHERE SAL >= 1000 AND SAL <= 2000

4. What is the difference between IN and multiple OR conditions?

Answer:

- Both give the same result, but IN is shorter and more readable.

Example

WHERE DEPTNO IN (10,20,30)

is same as

WHERE DEPTNO = 10 OR DEPTNO = 20 OR DEPTNO = 30;

In interviews: Always say **IN is better for readability and maintenance**.

5. What does IS NULL do, and why can't we use = NULL?

Answer:

- NULL means unknown or missing, not zero/blank.
- = NULL won't work because NULL cannot be compared with equality operators.

Correct way:

```
WHERE COMM IS NULL
```

6. Real-world use of LIKE operator?

Answer:

- Used for pattern matching in search.
- Example: Search all emails ending with @gmail.com.

```
WHERE Email LIKE '%@gmail.com'
```

Industry use: Login validations, reporting, data cleanup.

Functions

What are Functions in SQL?

- A **function** is like a ready-made mini-program inside SQL that **takes input, performs a task, and returns a value**.
- Functions save time because you don't need to write logic again and again.

Real world example:

- Instead of writing a formula to calculate an employee's name length, you just use LEN(Name).
- Instead of coding date math, use DATEDIFF to quickly find employee tenure.

Why do we need Functions?

- To **transform data** quickly (uppercase, formatting, date math).
- To **make reports readable** (round salaries, format dates).
- To **simplify logic** → instead of long queries, use small functions.
- To **improve performance** → many functions are optimized internally.

Types of Functions

1. **System / Built-in Functions** – Provided by SQL Server.
    - Example: LEN(), ROUND(), GETDATE()
2. **User Defined Functions (UDFs)** – You create them when system functions are not enough.
    - Example: CREATE FUNCTION GetFullName(...)

Character / String Functions

System / Built in functions

1. **Character / String functions.**

- **Its a functions where it is working on the character and the strings.**
    - **LEN() → Returns length of string**
    - **SELECT ENAME,LEN(ENAME) AS NameLength FROM EMP;**
    - **Real world: Find employees whose name are too short/Long for ID cards.**
    - **SUBSTRING () → Extracts part of a string**
    - **SELECT ENAME, SUBSTRING(ENAME,1,3)As FirstThreeCharacters FROM EMP;**
    - **Many Real-world: (banking, telecom, legacy ERP) stores data as text for compatibility.**

- **REPLACE()→ Replace part of string**
- **SELECT ENAME, REPLACE(ENAME,'a','@') As MaskedName from EMP;**
- **Real world: Mask Sensitive info(replace digits in passport).**
- **UPPER() / LOWER()**
- **SELECT UPPER(ENAME), LOWER(ENAME) FROM EMP;**
- **Industry use: Standardize data before comparison/ Search.**

Numeric Functions

1. ROUND() → Round to n decimals

SELECT Salary, ROUND(Salary, 2) AS RoundedSalary

FROM Emp;

1234.567 → 1234.57

Use: Financial reports, invoices.

2. CEILING() → Round up

SELECT CEILING(Salary/1000.0) AS SalaryInThousands

FROM Emp;

3450 → 4 (thousands)

Use: Estimating project budgets.

3. FLOOR() → Round down

SELECT FLOOR(Salary/1000.0) AS SalaryInThousands

FROM Emp;

3450 → 3

Use: Billing cycles (round down to nearest day/hour).

4. ABS() → Absolute value

SELECT ABS(Salary - 5000) AS Difference

FROM Emp;

If Salary = 4800 → 200

Use: Error calculation, deviation analysis.

What does ABS() mean?

- **ABS = Absolute value function**.
- It simply **removes the sign** (– or +) from a number and gives the magnitude.
- Example:
    - ABS(10) → 10
    - ABS(-10) → 10

So, ABS(x) always returns a **non-negative value**.

Date Functions

1. GETDATE() → Current system date & time

SELECT GETDATE() AS Today;

Use: Audit logging (last updated time).

2. DATEADD() → Add interval

SELECT Ename, HIREDATE, DATEADD(YEAR,1,HIREDATE)AS NextReview FROM emp;
Adds 1 year to hire date.

Use: Probation review, contract renewals.

3. DATEDIFF() → Difference between dates

SELECT ENAME,HIREDATE, DATEDIFF(YEAR, HIREDATE, GETDATE()) AS Experience FROM emp;

Returns years of service.

Use: Employee appraisal, tenure tracking.

4. DATEPART() → Extract part of date

SELECT ENAME,HIREDATE, DATEPART(MONTH, HIREDATE)As JoinMonth from emp;

If HireDate = 2023-05-15 → 5

Use: Find seasonal hiring trends.

Conversion Functions

1. CAST() → Convert data type

SELECT ENAME,sal, CAST(SAL AS VARCHAR)+' INR' AS SalaryWithcurrency from emp;

2. CONVERT() → Convert with style

SELECT ENAME, HIREDATE, CONVERT(VARCHAR, HIREDATE, 103) AS UKFORMAT FROM emp;

15-05-2023

Use: Regional date formatting (UK, US, India).

CONVERT(VARCHAR, HireDate, 103)

- CONVERT() changes a value from one data type to another.
- Here:
  - HireDate (a **DATE/DATETIME**) → converted into a **string (VARCHAR)**.
  - 103 → is a **style code** that tells SQL *how to format the date*.

Style Code 103

- 103 = **British/UK date format** → dd/MM/yyyy
- Example:
  - If HireDate = 2023-05-15 (database stores in yyyy-MM-dd format)
  - After conversion → "15/05/2023"

AS UKFormat

- This renames the output column to UKFormat for clarity.

INTERVIEW QUESTIONS

1. What are functions in SQL and why do we use them?

Answer:

- Functions are pre-built mini-programs in SQL.
- They take input, process it, and return a single value.
- Needed for formatting, calculations, reporting, and performance.

Example:

SELECT LEN(ENAME) FROM emp;

2. Difference between system functions and user-defined functions (UDFs)?

Answer:

- System functions: Predefined in SQL (LEN, ROUND, GETDATE).

- UDFs: Created by users when system functions are not enough.
  Example: CREATE FUNCTION fn_GetFullName(...)

3. What's the difference between LEN() and SUBSTRING() functions?

Answer:

- LEN() returns the length of a string.
- SUBSTRING() extracts part of a string.

Example:

LEN('Karthik') → 7

SUBSTRING('Karthik', 1, 3) → Kar

Industry use: Cleaning names, extracting codes from IDs.

4. Real-world example of REPLACE()?

Answer:

- Used to mask sensitive information.

SELECT REPLACE(Phone, '123', 'XXX') FROM Customers;

Use: Data privacy, compliance (mask PAN, Aadhar, Phone numbers).

5. Explain ROUND, CEIL, FLOOR with real-world examples.

Answer:

- ROUND(1234.567, 2) → 1234.57 → Use in invoices.
- CEILING(3450/1000.0) → 4 → Round UP to nearest 1000 for budgets.
- FLOOR(3450/1000.0) → 3 → Round DOWN to nearest 1000 for billing.

6. Difference between GETDATE() and DATEDIFF()?

Answer:

- GETDATE() → current system date & time.
- DATEDIFF() → calculates gap between two dates.
  Example:

  DATEDIFF(YEAR, HireDate, GETDATE()) AS Experience

Use: Finding employee tenure for appraisal.

7. When do we use CAST vs CONVERT?

Answer:

- Both change data type.
- CAST: ANSI standard, portable.
- CONVERT: SQL Server-specific, supports style formatting (e.g., date formats). Example:

CAST(SAL AS VARCHAR)

CONVERT(VARCHAR, HireDate, 103) → dd-mm-yyyy format

Aggregate (Grouping) Functions in SQL

1. What are Aggregate Functions?

- Aggregate functions **perform calculations on a set of rows** and return a **single summarized value**.
- They are also called **grouping functions** because they are often used with GROUP BY to summarize data by groups (e.g., per department).

Think of aggregate functions as the "Excel SUM, AVERAGE, COUNT" but inside SQL for databases.

2. Why do we need Aggregate Functions?

- To get insights from large datasets.
- To summarize and report data instead of checking row by row.
- Industry examples:
  - Calculate **average salary** in each department.
  - Find the **highest transaction amount** in banking.
  - Get the **total sales** for a product.
  - Count the **number of students** enrolled in a course.

Keywords with Explanation

1. **SUM()** – Adds all numeric values.
   Example: SUM(Salary) → total salary bill of employees.
2. **AVG()** – Returns average of numeric values.
   Example: AVG(Salary) → average salary in a department.
3. **COUNT()** – Counts number of rows (or non-null values in a column).
   Example: COUNT(*) → number of employees.
4. **MAX()** – Returns the maximum value.
   Example: MAX(Salary) → highest paid employee's salary.
5. **MIN()** – Returns the minimum value.
   Example: MIN(HireDate) → earliest hire date.

◆ SUM()

- Adds up all values in a numeric column.

Example

-- Total salary of all employees

SELECT SUM(SAL) AS TotalSalary

FROM Emp;

Industry use: Find **total revenue**, **total expenses**, etc.

- **AVG()**
  - Calculates the average of a numeric column.

-- Average salary in company

SELECT AVG(SAL) AS AvgSalary

FROM Emp;

Industry use: Find **average transaction size**, **average marks**, **average response time**.

- **COUNT()**
  - Counts number of rows (or non-null values in a column).

-- Total employees

SELECT COUNT(*) AS TotalEmployees

FROM Emp;

-- Employees who earn commission

SELECT COUNT(COMM) AS EmployeesWithComm

FROM Emp;

Industry use: Count **active users**, **number of products sold**, **attendance in class**.

- **MIN()**
  - Finds the smallest value in a column.

-- Minimum salary

SELECT MIN(SAL) AS MinSalary

FROM Emp;

Industry use: Find **lowest score**, **minimum delivery time**, **cheapest price**.

- **MAX()**
  - Finds the largest value in a column.

-- Highest salary

SELECT MAX(SAL) AS MaxSalary

FROM Emp;

Industry use: Find **highest sales**, **maximum marks**, **fastest delivery time**.

Display maximum salary for deptno 10

SELECT MAX(SAL) AS MaxSalary FROM Emp WHERE DEPTNO=10;

Display maximum salary for deptno 20

SELECT MAX(SAL) AS MaxSalary FROM Emp WHERE DEPTNO=20;

Display maximum salary for deptno 30

SELECT MAX(SAL) AS MaxSalary FROM Emp WHERE DEPTNO=30;

SELECT MAX(SAL) AS MaxSalary FROM Emp WHERE DEPTNO in (10,20,30)

Note: It will not show the salary department wise, it will show the total maximum salary.

4. Using Aggregate Functions with GROUP BY

Aggregate functions are powerful when used with GROUP BY to summarize data per group.

-- Total salary by department

SELECT DEPTNO, SUM(SAL) AS TotalDeptSalary

FROM Emp

GROUP BY DEPTNO;

-- Average salary by job

SELECT JOB, AVG(SAL) AS AvgJobSalary

FROM Emp

GROUP BY JOB;

Display Job wise average salary, minimum salary, maximum salary, sumon salary, number of salary?

Select min(sal) AS MinSal,max(sal) AS MaxSal, avg(sal) AS AVGSal, sum(sal)AS SUMSAL, count(sal)AS CountSal, job from emp group by JOB;

Display job wise average salary only if average salary greater than 2000

Select avg(sal), job from emp group by job where avg(sal)> 2000;

Error: Incorrect syntax near the keyword 'where'.

Select avg(sal), job from emp where avg(SAL) > 2000 group by JOB;

Error: An aggregate may not appear in the WHERE clause unless it is in a subquery contained in a HAVING clause or a select list, and the column being aggregated is an outer reference.

Conditions to Apply Group by

- To apply the filter on grouping values we should not use where clause for that we have "having" clause.
- Where clause is used for applying the filter on non-grouping values.
- The columns which are selected must be in group by clause.
- Note: Grouping values mean either AVG, MIN, or MAX or SUM or COUNT.
- Non-grouping values mean ex: sal > or Job or Deptno in etc.

select avg(sal), job from emp group by job having avg(sal)> 2000;

SQL: GROUPING, HAVING & SELECT Flow

---

1. The SELECT Statement Flow

- **Syntax order (what you write):**
  SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ...

2. Row-level vs Group-level Data

- **Row-level (detail data):** Columns directly from the table like ENAME, SAL, DEPTNO.
- **Group-level (aggregated data):** Results after grouping like AVG(SAL), SUM(SAL).

Rule: In a grouped query, everything you SELECT must be either:

- in the GROUP BY clause, OR

- an aggregate function.

Otherwise, SQL will throw:

"Column ... is invalid in the select list because it is not contained in either an aggregate function or the GROUP BY clause."

4. WHERE vs HAVING

- **WHERE → filters rows before grouping** (works on detail data).
  Example: Only consider employees with salary > 1000.
- **HAVING → filters groups after grouping** (works on aggregated data).
  Example: Only show departments whose average salary > 2000.

Memory tip:

- WHERE = rows
- HAVING = groups

5. GROUP BY Essentials

- All non-aggregated columns in SELECT must appear in GROUP BY.
- You can group by multiple columns (e.g., DEPTNO, JOB).
- Aggregate functions ignore NULLs (except COUNT(*), which counts all rows).

6. ORDER BY in Grouped Queries

- You can sort by any column, aggregate, or even alias.
- Example: ORDER BY SUM(SAL) DESC → highest total salary departments on top.

Mastering WHERE, HAVING, GROUP BY, ORDER BY

Example 1: Filtering with WHERE before grouping

Problem: Show departments with employees earning more than 2000.
Explanation: Here we filter individual rows before grouping.

```
SELECT DEPTNO, ENAME, SAL

FROM Emp

WHERE SAL > 2000;
```

Insight: WHERE always filters row-level data (non-aggregates).

Example 2: Filtering groups with HAVING

Problem: Show departments where the **average salary** is above 2500.
Explanation: You cannot use WHERE AVG(SAL) because aggregates come *after* grouping → use HAVING.

```
SELECT DEPTNO, AVG(SAL) AS AvgSalary

FROM Emp

GROUP BY DEPTNO

HAVING AVG(SAL) > 2500;
```

Insight: HAVING is specifically for **aggregate conditions**.

Example 3: WHERE + HAVING together

Problem: Count employees per department, but only consider salaries > 1000. Then show departments having at least 3 employees.

```
SELECT DEPTNO, COUNT(*) AS EmpCount

FROM Emp

WHERE SAL > 1000      -- filters rows first

GROUP BY DEPTNO

HAVING COUNT(*) >= 3;   -- filters groups after aggregation
```

Insight: Order of evaluation: WHERE filters rows → GROUP BY buckets → HAVING filters buckets.

	DEPTNO	EmpCount
1	10	3
2	20	4
3	30	5

Example 4: GROUP BY multiple columns

Problem: Find average salary per department and job.

```
SELECT DEPTNO, JOB, AVG(SAL) AS AvgSalary

FROM Emp

GROUP BY DEPTNO, JOB

ORDER BY DEPTNO, AvgSalary DESC;
```

Insight: Multiple columns in GROUP BY create smaller buckets (Dept+Job).

	DEPTNO	JOB	AvgSalary
1	10	PRESIDENT	5000
2	10	MANAGER	2450
3	10	CLERK	1300
4	20	ANALYST	3000
5	20	MANAGER	2975
6	20	CLERK	950
7	30	MANAGER	2850
8	30	SALESMAN	1400
9	30	CLERK	950

Example 5: ORDER BY aggregates

Problem: Show departments sorted by **total salary** (highest first).

```
SELECT DEPTNO, SUM(SAL) AS TotalSalary
FROM Emp
GROUP BY DEPTNO
ORDER BY TotalSalary DESC;
```

Insight: You can order results using aggregates or aliases.

Example 6: Classic Error — non-grouped column

Problem: Show department with employee names and average salary.
Wrong:

```
SELECT DEPTNO, ENAME, AVG(SAL)
FROM Emp
GROUP BY DEPTNO;
```

Error: ENAME is not in GROUP BY or aggregated.
Fix: Use STRING_AGG or COUNT instead.

```
SELECT DEPTNO, STRING_AGG(ENAME, ', ') AS Employees, AVG(SAL) AS AvgSalary
FROM Emp
GROUP BY DEPTNO;
```

Results | Messages

	DEPTNO	Employees	AvgSalary
1	10	CLARK, KING, MILLER	2916
2	20	FORD, ADAMS, SCOTT, SMITH, JONES	2175
3	30	MARTIN, BLAKE, ALLEN, WARD, TURNER, JAMES	1566

What STRING_AGG(ENAME, ', ') does:

- STRING_AGG is an **aggregate function** (like SUM, AVG, MAX, etc.).
- Instead of doing math, it **concatenates values from multiple rows into a single string**.
- ENAME is the column being aggregated (employee names).
- The second argument ', ' is the **separator** to place between each value.

So for each department (DEPTNO), it will:

- Collect all employee names (ENAME)
- Join them together into **one single string**
- Separate each name with a comma and a space.

Example 9: HAVING without WHERE

Problem: Show departments that have more than 4 employees (any salary).

```
SELECT DEPTNO, COUNT(*) AS EmpCount
FROM Emp
GROUP BY DEPTNO
HAVING COUNT(*) > 4;
```

Insight: If no row-level filter is needed, go straight to HAVING.

Example 10: Combining all — real business query

Problem: Show departments where the total salary > 3000 but only employees with salary > 1000 sort by total salary descending order.

```
SELECT DEPTNO, SUM(SAL) AS TotalSalary, COUNT(*) AS EmpCount
FROM emp
WHERE SAL > 1000
GROUP BY DEPTNO
```

HAVING SUM(SAL) >3000

ORDER BY TotalSalary DESC

Insight: This is a full SELECT flow: WHERE → GROUP BY → HAVING → ORDER BY

	DEPTNO	TotalSalary	EmpCount
1	20	8975	3
2	10	7450	2
3	30	4450	2

SQL Subqueries

Introduction:

- Query within a query is called as Subquery.
- The output of inner query becomes input to the outer query.
- To write a subquery there should be a common column or reference column in both tables.

Display the employees who are from Dallas or Chicago ?

Step 1: Read the query 2 or 3 times

Display the employees who are from Dallas

Step 2: Identify the how many tables required

EMP, DEPT

Step 3: Identify outer table & inner table

Outer table= Emp

Inner table = dept

Step 4: Identify the inner table column with condition

Loc = 'Dallas'

Step 5: Solve inner query

Select * from dept where loc = 'Dallas'

Step 6: Identify the common column/Reference column b/w 2 tables dept no

Deptno

Step 7: Solve outer query

Select * from emp where deptno

Step 8: Combine outer query & inner query

Select * from emp

Where deptno = (Select * from dept where loc = 'Dallas');

Error message:

Step 9:

Select * from emp

Where deptno = (select deptno from dept where loc = 'DALLAS')

Display the department information for smith ?

Tables: DEPT, EMP

Outer Table: DEPTNO

INNER Table: EMPNO

CONDITION ENAME=’SMITH’

```
SELECT * from dept
        Where deptno=(SELECT deptno from emp where ename='SMITH')
```

Display the employees who are earning more than smith ?

```
SELECT * FROM EMP
        WHERE SAL > (SELECT SAL FROM EMP WHERE ENAME=’SMITH’)
```

Display the employees who are not from DALLAS and he should be a manager

```
SELECT * FROM emp
        WHERE DEPTNO = (SELECT DEPTNO FROM DEPT WHERE LOC NOT IN ('DALLAS'));
```

ERROR MESSAGE:
Subquery returned more than 1 value. This is not permitted when the subquery follows =, !=, <, <= , >, >=

Because we are using ‘=’ sign which returns only one value but we need three values so go for in operator

```
SELECT * FROM emp
        WHERE DEPTNO in(SELECT DEPTNO FROM DEPT WHERE LOC NOT IN ('DALLAS')) AND JOB='MANAGER';
```

Display the employees who are joined before Jones ?

```
SELECT * FROM EMP
        WHERE HIREDATE<(SELECT HIREDATE FROM EMP WHERE ENAME='JONES')
```

Display department information for all the managers ?

Outer table: dept

Inner table: emp

SELECT * FROM DEPT

WHERE DEPTNO IN(SELECT DEPTNO FROM EMP WHERE JOB='MANAGER')

Display the 3rd maximum salary employ details ?

SELECT * FROM emp WHERE sal < (SELECT MAX(sal) FROM emp WHERE sal < (SELECT MAX(sal) FROM emp));

Types of Subqueries

a) Single-row Subqueries

These return exactly one row. Used with operators like =, >, <.

Example:

SELECT * FROM emp WHERE sal = (SELECT MAX(sal) FROM emp);

b) Multi-row Subqueries

These return multiple rows. Used with operators like IN, ANY, ALL.

Example using IN:

SELECT * FROM emp WHERE deptno IN (SELECT deptno FROM dept WHERE loc NOT IN ('DALLAS'));

c) Nested Subqueries

Subqueries inside subqueries. Useful for problems like 2nd max, 3rd max salary.

Example:

SELECT * FROM emp WHERE sal < (SELECT MAX(sal) FROM emp WHERE sal < (SELECT MAX(sal) FROM emp));

Display empno, ename, deptno, location for all the employees who are from dallas

SELECT empno, ename, deptno, loc from emp

WHERE DEPTNO IN (SELECT DEPTNO FROM DEPT where loc in (‘DALLAS’)):

Invalid column name 'loc'.

Why does this happen?

1. **Look at the FROM clause**
   → FROM emp
   This means the **outer query is only looking at the EMP table**.
2. **What columns does EMP have?**
   - empno, ename, job, mgr, hiredate, sal, comm, deptno
     ❌ It does **not** have LOC.
3. **But we wrote SELECT … loc**
   - Here, SQL looks inside EMP for a column called LOC.
   - It doesn't find it → error.

In the above query we are trying to display employ details and departmental details.

At any point of time using sub queries we can display only one table information based on one or more than one table conditions.

SQL Joins

1. What are Joins in SQL?

- A **Join** in SQL is used to combine data from **two or more tables** based on a related column (usually a **primary key** in one table and a **foreign key** in another).
- Think of it as "bringing together" details from different tables into one report.

Example:

- **EMP** table stores employee details (Empno, Ename, Job, Deptno).
- **DEPT** table stores department details (Deptno, Dname, Loc).
- Using a join, we can show: *"Employee name with their department name and location."*

2. Why do we need Joins?

In real-world databases, data is **normalized** (split into multiple tables for efficiency and to avoid duplication).

- **EMP** only stores the department number (DEPTNO), not the department name/location.
- **DEPT** stores department details.
- Without joins, we would only see numbers (DEPTNO = 10, 20...) instead of meaningful info like "ACCOUNTING in NEW YORK".

Joins solve the problem: They **combine related data** across tables to answer business questions.

Real world: In a company database, **EMP** table has employee info and **DEPT** table has department info.
If a manager asks:

"Show me all employees along with their department names,"
you need to pull data from **both tables** → that's where JOIN comes in.

What is an INNER JOIN?

- An **INNER JOIN** returns only the rows that have **matching values in both tables**.
- In our case: Only employees whose Deptno exists in the DEPT table will be shown.
- If there is no matching department, that employee won't appear in the result.

Syntax of INNER JOIN

```
SELECT e.Empno, e.Ename, e.Deptno, d.Dname, d.Loc
FROM Emp e
INNER JOIN Dept d
ON e.Deptno = d.Deptno;
```

Breaking it Down

a) Aliases (e and d)

- Emp e → nickname e for EMP table.
- Dept d → nickname d for DEPT table.

Instead of writing Emp.Ename, we can just write e.Ename.

This is **shorter and clearer**, especially when joining multiple tables.

Why e.Deptno and d.Deptno?

- Both EMP and DEPT tables have a column called Deptno.
- To avoid confusion, we must **prefix column names** with their table alias.
    - e.Deptno → Deptno from EMP
    - d.Deptno → Deptno from DEPT

The JOIN condition: ON e.Deptno = d.Deptno

- This is the **rule** that tells SQL how to connect the two tables.
- It matches employees to their departments by department number.

Without this condition, SQL would try to join **every employee with every department** (Cartesian product), which makes no sense.

Example Output:

If EMP table looks like:

Empno	Ename	Deptno
101	Smith	10
102	Allen	20
103	Clark	30

And DEPT table looks like:

Deptno	Dname	Loc
10	ACCOUNT	NEW YORK
20	SALES	CHICAGO
40	HR	BOSTON

The result of INNER JOIN will be:

Empno	Ename	Deptno	Dname	Loc
101	Smith	10	ACCOUNT	NEW YORK
102	Allen	20	SALES	CHICAGO

Notice:

- Clark (Deptno 30) doesn't show up because Deptno 30 doesn't exist in DEPT table.
- Deptno 40 (HR) doesn't show up because no employee belongs to it.

Real-World Use Cases of INNER JOIN

- HR reports: List employees with their department names.
- Sales dashboards: Show sales transactions with customer details.
- Banking: Show accounts with their branch details.
- Education: Show students with their enrolled course names.

Note: INNER JOIN returns only matching rows between two tables, based on the join condition.

LEFT JOIN (LEFT OUTER JOIN) in SQL

1. What is a LEFT JOIN?

- A **LEFT JOIN** returns:
    1. **All rows from the left table (first table you mention in the query)**

2. Plus the **matching rows from the right table** (if any)
3. If there's **no match**, you still get the row from the left table but the right-side columns will show **NULL**.

Think of it like: "Give me all employees (left table), and add their department details if available. If they don't belong to a valid department, still list them, but dept info will be NULL."

2. Syntax

```
SELECT e.Empno, e.Ename, e.Deptno, d.Dname, d.Loc
FROM Emp e
LEFT JOIN Dept d
ON e.Deptno = d.Deptno;
```

3. Breaking it Down

a) LEFT table vs RIGHT table

- Emp e = LEFT table (we are selecting from this first).
- Dept d = RIGHT table.

LEFT JOIN = keep everything from left table.

b) Condition: ON e.Deptno = d.Deptno

- Matches employees with departments using Deptno.
- If match found → department details appear.
- If no match → employee still shows up, but dept columns = NULL.

Example Output:

EMP Table

Empno	Ename	Deptno
101	Smith	10
102	Allen	20
103	Clark	30
104	Scott	50

DEPT Table

Deptno	Dname	Loc
10	ACCOUNT	NEW YORK
20	SALES	CHICAGO
40	HR	BOSTON

5. LEFT JOIN Result

Empno	Ename	Deptno	Dname	Loc
101	Smith	10	ACCOUNT	NEW YORK
102	Allen	20	SALES	CHICAGO
103	Clark	30	NULL	NULL
104	Scott	50	NULL	NULL

Key points:

- Clark (30) and Scott (50) are still listed, even though their departments don't exist.
- That's why Dname and Loc show as **NULL**.
- This is the **main difference** from INNER JOIN (which would have excluded Clark and Scott).

Real-World Use Cases of LEFT JOIN

- Show **all employees**, even if some don't have a department assigned yet.
- Show **all customers**, even if some never placed an order.
- Show **all products**, even if some products have never been sold.
- In data cleaning: identify rows in left table that don't have matching records in right table (because NULLs highlight missing matches).

Note: LEFT JOIN returns all rows from the left table, along with matching rows from the right table. If there is no match, NULLs are returned for the right table's columns.

RIGHT JOIN (RIGHT OUTER JOIN) in SQL

1. **What is a RIGHT JOIN?**

- A **RIGHT JOIN** returns:
    1. **All rows from the right table** (the second table in your query)
    2. Plus the **matching rows from the left table** (if any)
    3. If there's **no match**, you still get the row from the right table but the left-side columns will show NULL.

Think of it like: "Give me all departments (right table), and add their employees if available. If no employees are in that department, still show the department, but employee details will be NULL."

2. Syntax

```
SELECT e.Empno, e.Ename, e.Deptno, d.Dname, d.Loc

FROM Emp e

RIGHT JOIN Dept d

ON e.Deptno = d.Deptno;
```

Breaking it Down

a) LEFT table vs RIGHT table

- Emp e = LEFT table (employees).
- Dept d = RIGHT table (departments).

RIGHT JOIN = keep everything from right table.

b) Condition: ON e.Deptno = d.Deptno

- Matches employees with departments using Deptno.
- If match found → employees show under that dept.
- If no match → department still appears, but employee details = NULL.

Example output:

EMP Table

Empno	Ename	Deptno
101	Smith	10
102	Allen	20
103	Clark	30

DEPT Table

Deptno	Dname	Loc
10	ACCOUNT	NEW YORK
20	SALES	CHICAGO
40	HR	BOSTON

5. RIGHT JOIN Result

Empno	Ename	Deptno	Dname	Loc
101	Smith	10	ACCOUNT	NEW YORK
102	Allen	20	SALES	CHICAGO
NULL	NULL	40	HR	BOSTON

Key points:

- HR (Dept 40) still appears, even though no employees belong to it.
- That's why Empno, Ename, Deptno (from Emp) = NULL.
- This is the main difference from INNER JOIN (which would have excluded Dept 40).

Real-World Use Cases of RIGHT JOIN

- Show all departments, even if no employees are assigned to them.
- Show all products, even if nobody purchased them.
- Show all projects, even if no employees are currently allocated.
- Useful in reporting when you care about the right-side entity coverage (e.g., every department must be listed).

Notes: RIGHT JOIN returns all rows from the right table, along with matching rows from the left table. If there is no match, NULLs are returned for the left table's columns.

FULL OUTER JOIN in SQL

1. What is a FULL OUTER JOIN?

- A **FULL OUTER JOIN** returns:
    1. **All rows from the left table (Emp)**
    2. **All rows from the right table (Dept)**
    3. Matching rows where possible
    4. If there's **no match**, SQL fills with **NULLs** for the missing side

Think of it like: "Show me all employees and all departments, even if some employees don't belong to any department or some departments don't have any employees."

2. Syntax

```
SELECT e.Empno, e.Ename, e.Deptno, d.Dname, d.Loc
FROM Emp e
FULL OUTER JOIN Dept d
ON e.Deptno = d.Deptno;
```

3. Breaking it Down

- Emp e = LEFT table
- Dept d = RIGHT table
- FULL OUTER JOIN = Keep everything from both tables
- ON e.Deptno = d.Deptno = match employees to their departments

If match → data merged
If no match → NULLs fill in missing side

Example output:

EMP Table

Empno	Ename	Deptno
101	Smith	10
102	Allen	20
103	Clark	30
104	John	NULL

DEPT Table

Deptno	Dname	Loc
10	ACCOUNT	NEW YORK
20	SALES	CHICAGO
40	HR	BOSTON

5. FULL OUTER JOIN Result

Empno	Ename	Deptno	Dname	Loc
101	Smith	10	ACCOUNT	NEW YORK
102	Allen	20	SALES	CHICAGO
103	Clark	30	NULL	NULL
104	John	NULL	NULL	NULL
NULL	NULL	40	HR	BOSTON

Key Observations

- Combines LEFT JOIN + RIGHT JOIN results
- Shows:
  - Matched records
  - Unmatched records from LEFT table (Emp)
  - Unmatched records from RIGHT table (Dept)
- Ensures no data is lost from either side

Real-World Use Cases

- Show all employees and all departments, even if some are unassigned.

- Show all customers and all orders, including customers who didn't order or orders with no assigned customer (data errors).
- Show all students and all courses, even if a student isn't enrolled or a course has zero students.

Note: FULL OUTER JOIN returns all rows from both tables. Matched rows appear together, unmatched rows show with NULLs.

CROSS JOIN in SQL

1. What is a CROSS JOIN?

- A **CROSS JOIN** returns the **Cartesian product** of two tables.
- This means **every row from the first table is combined with every row from the second table.**
- There's **no join condition** required.

Think of it like: "Match everything with everything."

2. Syntax

```
SELECT e.Empno, e.Ename, d.Deptno, d.Dname

FROM Emp e

CROSS JOIN Dept d;
```

3. Breaking it Down

- If **Emp has 5 rows** and **Dept has 3 rows**,
  then CROSS JOIN will produce **5 × 3 = 15 rows**.
- No ON condition here. It just multiplies the rows.

Empno	Ename
101	Smith
102	Allen

DEPT Table

Deptno	Dname
10	ACCOUNT
20	SALES
30	HR

5. CROSS JOIN Result (2 × 3 = 6 rows)

Empno	Ename	Deptno	Dname
101	Smith	10	ACCOUNT
101	Smith	20	SALES
101	Smith	30	HR
102	Allen	10	ACCOUNT
102	Allen	20	SALES
102	Allen	30	HR

Key Observations

- **No matching condition** → It blindly pairs all rows.
- Useful for **generating combinations**, not for normal reporting.
- Can generate **huge results** (be careful with large tables).

Real-World Use Cases

- **Test Data Generation:**
  E.g., create all possible combinations of dates × products × stores for reporting.
- **Business Scenarios:**
  - List all employees against all shifts (before actual assignment).
  - List all students against all subjects (before enrollment).

Notes: CROSS JOIN returns the Cartesian product of two tables — all combinations of rows.

Correlated Subqueries in SQL

1. What is a Subquery? (Quick Recap)

A **subquery** is a query inside another query.

Example

```
SELECT Ename, Deptno

FROM Emp

WHERE Deptno IN (SELECT Deptno FROM Dept WHERE Loc = 'NEW YORK');
```

Here, the inner query runs first → finds all departments in NEW YORK.
Then, the outer query fetches employees belonging to those departments.

This is a **simple (non-correlated)** subquery because the inner query runs **independently** — it doesn't rely on the outer query's data.

2. What is a Correlated Subquery?

A **correlated subquery** is one where the **inner query depends on the outer query** — it uses a value from the outer query.

Think of it like:

"For each row in the outer query, run the inner query to check something."

So the inner query is **executed once for every row** of the outer query.

3. Why Do We Use Correlated Subqueries?

Because sometimes, you need to compare **a row against data that depends on that same row**.

Examples:

- Find employees who earn more than the **average salary of their department**.
- Find customers whose order total is **above their own average spend**.
- Find students who scored **above their class average**.

Joins are great for combining tables;
but correlated subqueries shine when you need **row-by-row contextual comparisons**.

4. Syntax of a Correlated Subquery

```
SELECT e.Empno, e.Ename, e.Sal, e.Deptno
FROM Emp e
WHERE e.Sal > (
  SELECT AVG(Sal)
  FROM Emp
  WHERE Deptno = e.Deptno
);
```

Explanation:

- Outer query → selects employee info.
- Inner query → calculates average salary **of the same department** as each employee.
- WHERE Deptno = e.Deptno makes it correlated → the inner query depends on each row of e in the outer query.

5. Understanding Execution Flow

Outer query picks one employee (say, ALLEN, Deptno = 30).
Inner query calculates the average salary for Deptno = 30.
Outer query compares ALLEN's salary with that average.
Repeats this for every employee.

This is why correlated subqueries are **slower** than normal subqueries — the inner query runs once per row.

7. EXISTS, IN, ANY, and ALL — Explained Simply

Let's understand these four powerful operators — they often appear with subqueries (especially correlated ones).

EXISTS

Used to check if the subquery **returns at least one row**.

```
SELECT Ename, Deptno
FROM Emp e
WHERE EXISTS (
  SELECT 1
  FROM Dept d
  WHERE d.Deptno = e.Deptno
  AND d.Loc = 'NEW YORK'
);
```

Meaning:
Show employees whose department exists in NEW YORK.

EXISTS returns TRUE if the subquery finds *even one match*.

Real-World Example:
Show all customers who have made at least one order.

IN

Used to check if a value **matches any value** returned by the subquery.

```
SELECT Ename, Deptno
FROM Emp
WHERE Deptno IN (
  SELECT Deptno FROM Dept WHERE Loc = 'NEW YORK'
);
```

Meaning:
Show employees who belong to any department located in NEW YORK.

IN → Subquery runs once.
EXISTS → Often runs for each row (correlated).

Use **EXISTS** when checking existence, **IN** when comparing values.

ANY (or SOME)

Used to compare a value against **any** value from the subquery.

```
SELECT Ename, Sal
FROM Emp
WHERE Sal > ANY (
   SELECT Sal FROM Emp WHERE Job = 'CLERK'
);
```

Meaning:
Show employees whose salary is greater than *at least one clerk*.

ANY → "At least one match satisfies the condition."

ALL

Used to compare a value against all values returned by the subquery.

```
SELECT Ename, Sal
FROM Emp
WHERE Sal > ALL (
   SELECT Sal FROM Emp WHERE Job = 'CLERK'
);
```

Meaning:
Show employees whose salary is higher than *every clerk's salary*.

ALL → "Must satisfy condition for every value in subquery."

8. Real-World Use Cases

Business Need	SQL Concept Used	Example
Find employees who earn more than average of their department	Correlated Subquery	WHERE e.Sal > (SELECT AVG(Sal) FROM Emp WHERE Deptno = e.Deptno)
Find employees who belong to departments in NEW YORK	IN	WHERE Deptno IN (SELECT Deptno FROM Dept WHERE Loc='NEW YORK')
Find employees if their department exists in DEPT table	EXISTS	WHERE EXISTS (SELECT 1 FROM Dept WHERE Deptno=e.Deptno)
Find employees who earn more than **any** clerk	ANY	WHERE Sal > ANY (SELECT Sal FROM Emp WHERE Job='CLERK')
Find employees who earn more than **all** clerks	ALL	WHERE Sal > ALL (SELECT Sal FROM Emp WHERE Job='CLERK')

9. Performance Tip

- Correlated subqueries are powerful but slower — they run the inner query for each row.
- If possible, **rewrite them using JOINs** for better performance in large data sets.
- Use **EXISTS** when you only need to check for existence — it stops after the first match, making it efficient.

10. Summary in One Line

"When a subquery must look back at the outer query for each row — that's a **correlated subquery**.
And when you want to control how you compare that data — you use **EXISTS, IN, ANY, or ALL.**"

What is Normalization?

Definition

Normalization is the process of **organizing data in a database** to reduce redundancy (duplicate data) and ensure **data integrity** (accuracy and consistency).

It's like cleaning your room — everything has a place, nothing is duplicated, and it's easier to find and update things later.

Why We Need Normalization

When databases grow large:

- Duplicate data wastes storage.
- Updates become inconsistent (same data repeated in multiple places).
- It's hard to maintain integrity across tables.

Normalization ensures:

- Data is stored **once**.
- Relationships are maintained via **keys** (Primary, Foreign).
- Updates are clean and reliable.

Think of a Real-World Example

Let's say we store employees and departments:

EmpID	EmpName	DeptName	DeptLoc
1	John	Sales	London
2	Mary	Sales	London
3	Raj	HR	Delhi
4	Tina	HR	Delhi

Problem:

- DeptName and DeptLoc are repeated for every employee.
- If "Sales" moves to "Manchester," you must update multiple rows.
- If one row is missed → data inconsistency.

Solution: Normalize.

Normal Forms (1NF → 3NF → BCNF)

FIRST NORMAL FORM (1NF)

Problem

When a column holds multiple values or repeating groups instead of one.

Before (Not in 1NF)

StudentID	StudentName	PhoneNumbers
101	John	9876543210, 9123456789
102	Mary	9988776655

Problem Explanation:

- The column PhoneNumbers has **multiple values** for the same student.
- If John changes one phone number, we have to edit a part of that text manually.
- Searching or filtering by phone number becomes difficult.

After (In 1NF)

StudentID	StudentName	PhoneNumber
101	John	9876543210
101	John	9123456789
102	Mary	9988776655

Fix Explanation:

- We separated each phone number into its own row.
- Every cell now contains only one value → atomic (indivisible).
- Easy to search, filter, and update.

SECOND NORMAL FORM (2NF)

Rule

- Must be in 1NF.
- Every non-key column must depend on the whole key, not part of it.

Before (Not in 2NF)

StudentID	CourseID	StudentName	CourseName
101	SQL101	John	SQL Basics
101	PY201	John	Python Intro
102	SQL101	Mary	SQL Basics

Key = (StudentID, CourseID) → both together make a unique record.

Problem Explanation:

- StudentName depends only on StudentID.
- CourseName depends only on CourseID.
- So these columns depend on **part of the key,** not the whole key.
  This causes **duplication** and **update issues.**

  If John changes his name spelling → you must update multiple rows.

After (In 2NF)

Student Table

StudentID	StudentName
101	John
102	Mary

Course Table

CourseID	CourseName
SQL101	SQL Basics
PY201	Python Intro

Enrollment Table

StudentID	CourseID
101	SQL101
101	PY201
102	SQL101

Fix Explanation:

- We split data into three tables.

- Each table focuses on one entity (Student, Course, Enrollment).
- Now no partial dependency → each non-key column depends fully on its key.

THIRD NORMAL FORM (3NF)

Rule

- Must be in 2NF.
- No transitive dependency — i.e., non-key column shouldn't depend on another non-key column.

Before (Not in 3NF)

EmpID	EmpName	DeptID	DeptName
1	John	D01	HR
2	Mary	D02	Finance
3	Raj	D01	HR

Problem Explanation:

- DeptName depends on DeptID, not directly on EmpID.
- This creates duplication — "HR" repeated for every employee in that department.
- If HR department name changes → must update many rows.

After (In 3NF)

Employee Table

EmpID	EmpName	DeptID
1	John	D01
2	Mary	D02
3	Raj	D01

Department Table

DeptID	DeptName
D01	HR
D02	Finance

Fix Explanation:

- We separated department details into a new table.
- Now DeptName is stored once, linked via DeptID.
- Data is clean, no redundancy, and easy to update.

BOYCE–CODD NORMAL FORM (BCNF)

Rule

- Stronger version of 3NF.
- For every relationship X → Y, X must be a **candidate key**.

Before (Not in BCNF)

Instructor	Course	Room
Smith	Database	R1
Jones	Python	R2
Smith	Database	R3

Assume:

- Each course is taught by one instructor (Instructor → Course).
- Each course happens in one room (Course → Room).

Problem Explanation:

- Instructor determines Course, but Instructor is not a key — violates BCNF.
- Same instructor can't teach two courses easily — data inconsistency possible.

After (In BCNF)

Instructor Table

Instructor	Course
Smith	Database
Jones	Python

Course Table

Course	Room
Database	R1
Python	R2

Fix Explanation:

- Split the table so that **every determinant** (Instructor, Course) is a key in its own table.
- Now data is consistent and reliable.

📄 **Summary Table**

Normal Form	Problem Type	Example Issue	Solution
1NF	Multiple values in one column	Phones = '9876,9123'	Split into separate rows (atomic values)
2NF	Non-key depends on part of key	StudentName depends only on StudentID	Split into Student, Course, Enrollment tables
3NF	Non-key depends on another non-key	DeptName depends on DeptID	Separate Department table
BCNF	Determinant not a candidate key	Instructor → Course but Instructor not key	Decompose into Instructor & Course tables

Denormalization (in Data Warehousing)

Definition

Denormalization = *intentionally combining normalized tables* to improve **query performance** in reporting or analytics systems.

Why Denormalize?

In OLTP (transactional systems):

- We normalize to reduce duplication and improve updates.

In OLAP / Data Warehousing:

- We **denormalize** to improve read performance.
- Queries join many tables → slow for analytics.
- By merging them, we **reduce joins** and **accelerate dashboards.**

Trade-off:
More redundancy → Faster reporting.
Less storage efficiency, but better performance.

Example:

Normalized structure (OLTP):

Customer(CustID, CustName)

Orders(OrderID, CustID, Amount)

To get total sales per customer:

```
SELECT c.CustName, SUM(o.Amount)
FROM Customer c JOIN Orders o
ON c.CustID = o.CustID
GROUP BY c.CustName;
```

For frequent reporting, instead of joining every time,
you can **create a denormalized table:**

CustID	CustName	OrderID	Amount
1	John	1001	200
1	John	1002	500
2	Mary	1003	700

Star Schema Example (Data Warehousing)

- **Fact Table:** Holds numeric, measurable data (Sales, Revenue).
- **Dimension Tables:** Describe context (Customer, Product, Time).

```
[DimCustomer]    [DimProduct]    [DimTime]
    \          |          /
     \         |         /
        [FactSales]
```

FactSales might include:

SaleID	CustKey	ProdKey	TimeKey	Amount

Benefits:

- Fewer joins.
- Simple for Power BI / Tableau.
- Fast aggregations.

When to Normalize vs Denormalize

Use Case	Approach	Why
Banking systems	Normalize	Frequent updates, data accuracy
E-commerce checkout	Normalize	Write-heavy
Reporting / Analytics	Denormalize	Read-heavy, speed matters
Data Warehouse	Denormalize	BI dashboards need quick reads

Real-World Examples

Domain	Use Case	Description
E-commerce	Denormalized "Sales Fact" for Power BI	Combines customer, product, and transaction info for dashboards
Banking	Normalized Account & Branch tables	Prevents redundant branch info
Retail Analytics	Star Schema	Quick monthly/quarterly sales reports
HR Analytics	Denormalized Employee Master for reporting	Combines dept, role, and salary for performance reports

Summary:
"Normalization is about *accuracy and consistency*.
Denormalization is about *speed and simplicity*.

You normalize to make data clean,
and denormalize to make it fast."

Views in SQL (Simple, Complex, and Materialized)

What is a View?

A **View** is like a **virtual table** — it doesn't store data by itself; it just saves a **query** that runs when you use it.

Think of a View as a **saved SELECT statement** that behaves like a table.

You can SELECT from a view just like you SELECT from a table.

Why We Use Views

- To simplify complex queries.
- To restrict access (show only part of a table).
- To re-use common queries across reports.
- To hide logic and keep the database secure and clean.

Simple View

Problem Without View

You have an Employee table like this:

EmpID	EmpName	Job	Salary	DeptID
1	John	Manager	5000	D01
2	Mary	Clerk	3000	D02
3	Raj	Analyst	4000	D01

Now, management says:

"Show only Employee Name, Job, and Salary — not Department details."

If you query directly, everyone needs to write:

```
SELECT EmpName, Job, Salary
FROM Employee;
```

If many reports use the same query, it's repetitive.
Also, if the HR team shouldn't see all data, you can't restrict easily.

Fix: Create a Simple View

```
CREATE VIEW Emp_Salary_View AS
SELECT EmpName, Job, Salary
FROM Employee;
```

Now the HR team can simply run:

```
SELECT * FROM Emp_Salary_View;
```

and see:

EmpName	Job	Salary
John	Manager	5000
Mary	Clerk	3000
Raj	Analyst	4000

Explanation

- A simple view is based on one table.
- It's read-only by default (unless you allow updates).
- You can grant permissions only on the view — securing sensitive data.

Complex View

Problem Without View

You have two tables:

Employee Table

EmpID	EmpName	Job	DeptID
1	John	Manager	D01
2	Mary	Clerk	D02
3	Raj	Analyst	D01

Department Table

DeptID	DeptName	Location
D01	HR	London
D02	Finance	Manchester

Managers want a combined report:

“Show Employee Name, Job, Department Name, and Location.”

You’d need to join every time:

```
SELECT e.EmpName, e.Job, d.DeptName, d.Location
FROM Employee e
JOIN Department d ON e.DeptID = d.DeptID;
```

Repeating this join in every report = messy and error-prone.

Fix: Create a Complex View

```
CREATE VIEW Emp_Dept_View AS
SELECT e.EmpName, e.Job, d.DeptName, d.Location
FROM Employee e
JOIN Department d ON e.DeptID = d.DeptID;
```

Now any user can just run:

```
SELECT * FROM Emp_Dept_View;
```

Output

EmpName	Job	DeptName	Location
John	Manager	HR	London
Mary	Clerk	Finance	Manchester
Raj	Analyst	HR	London

Explanation

- A complex view is built on multiple tables (using joins or aggregations).
- Used for reports, dashboards, and analytics.
- Read-only by nature — can't update through it easily.
- Simplifies access for analysts and BI tools (Power BI, Tableau, etc.).

Materialized View (Real Data Stored)

Problem With Normal Views

A normal view is virtual — data is not stored, only the query is.
So every time you select from it, SQL must recalculate the result.

If the query joins 5–6 big tables → slow performance for analytics or dashboards.

Fix: Use a Materialized View

Materialized View = View + Cached Data
It physically stores the result of the query for faster access.

Example:

```
CREATE MATERIALIZED VIEW Sales_Summary AS

SELECT Region, SUM(SalesAmount) AS TotalSales

FROM Sales

GROUP BY Region;
```

Data physically stored like this:

Region	TotalSales
North	120000
South	90000
East	110000

Later, you can refresh it to update:

REFRESH MATERIALIZED VIEW Sales_Summary;

Explanation

- Used in **data warehouses** and **Power BI reports**.
- Stores pre-calculated results for **speed**.
- Can be refreshed manually or automatically (daily/hourly).
- Great for dashboards where data doesn't change every minute.

Summary Table

Type of View	Description	Based On	Stores Data?	Use Case
Simple View	View from one table	Single table	No	Hide columns, simplify access
Complex View	View using joins/aggregations	Multiple tables	No	Reports, analytics, read-only
Materialized View	View with stored data	Any query	Yes	Fast reporting, dashboards

Key Takeaways

1. **Views** make data access simple and secure.
2. **Simple View** → single table, filtered columns.
3. **Complex View** → joins, aggregations, multiple tables.
4. **Materialized View** → physically stores results for faster reads.

5. Use views to **hide complexity** and **protect sensitive data** while still giving power to the users.

SQL Indexes — Clustered vs Non-Clustered (with Examples & Performance)

What is an Index?

An **index** in SQL is like the **index page of a book** —
it helps you find information *faster* without reading every page.

Without an index, SQL reads **every row** (called a "table scan").
With an index, SQL can **jump directly** to the rows it needs.

Problem Without an Index

Before — No Index (Slow Search)

Employee Table

EmpID	EmpName	Job	Salary	DeptID
1	John	Manager	8000	D01
2	Mary	Clerk	3000	D02
3	Raj	Analyst	4000	D01
4	Anil	Clerk	2500	D03
5	Priya	HR	3500	D02
...	...	...	...	...

Now imagine this table has **1 million rows.**

You run:

SELECT * FROM Employee WHERE EmpName = 'Raj';

What happens:

- SQL checks each row one by one (a full table scan).
- Very slow when the table is large.

After — Index Added (Fast Search)

We add an index:

CREATE INDEX idx_emp_name ON Employee(EmpName);

Now SQL doesn't scan all rows — it jumps directly to "Raj".

Result:

Search time goes from **seconds → milliseconds**.

Simple Explanation

- Think of the **index** like a sorted phone book.
- Without it → you read every name to find "Raj".
- With it → you jump to the "R" section instantly.

Clustered Index

What is It?

- **Physically orders** the data in the table itself.
- There can be **only one clustered index** per table because data can be physically sorted only one way.
- Usually created automatically on the **Primary Key**.

Before — No Order

EmpID	EmpName	Salary
3	Raj	4000
1	John	8000
5	Priya	3500
2	Mary	3000

If you search EmpID = 5, SQL must read line by line.

After — Clustered Index on EmpID

```
CREATE CLUSTERED INDEX idx_empid ON Employee(EmpID);
```

Now the table is **physically sorted** like this:

EmpID	EmpName	Salary
1	John	8000
2	Mary	3000
3	Raj	4000
5	Priya	3500

When you search for EmpID, SQL can use **binary search** (jumping directly to the location).

Real-World Analogy

Clustered Index = Phonebook arranged alphabetically.
The data *itself* is in sorted order → fast lookups by that key.

Before — Without Non-Clustered Index

Manager wants to find employees **by Job**:

SELECT * FROM Employee WHERE Job = 'Clerk';

SQL again scans every row → slow.

After — Non-Clustered Index

CREATE NONCLUSTERED INDEX idx_emp_job ON Employee(Job);

Now SQL uses the index (sorted list of Job titles) to quickly find all rows where Job = 'Clerk', then **jumps** to the actual data rows using pointers.

Job	Pointer
Analyst	→ row 3
Clerk	→ row 2
Clerk	→ row 4
HR	→ row 5
Manager	→ row 1

Real-World Analogy

Non-Clustered Index = Book index at the end —
it doesn't contain the pages themselves, just pointers to where the topic is.

Index Performance Comparison

Operation	Without Index	With Index
Searching by column value	Scans all rows (slow)	Jumps directly (fast)
Sorting	Needs to sort every time	Already sorted by index
INSERT/UPDATE	Fast	Slightly slower (index updates)
Storage	Less	More (extra structure)

When to Use Which

Type	Description	Best For	Count
Clustered Index	Physically sorts table data	Primary key, Range queries (BETWEEN, <, >)	Only 1
Non-Clustered Index	Stores pointers to data	Frequently searched columns (EmpName, Job)	Many allowed

Example in One Flow

Employee Table

```
CREATE TABLE Employee (
  EmpID INT PRIMARY KEY,        -- Clustered index by default
  EmpName VARCHAR(50),
  Job VARCHAR(30),
  Salary INT
);
```

Create Non-Clustered Index

```
CREATE NONCLUSTERED INDEX idx_emp_job ON Employee(Job);
```

Search Fast

SELECT EmpName, Salary FROM Employee WHERE Job = 'Clerk';

Summary Table

Concept	Clustered Index	Non-Clustered Index
Stores Data	Yes (reorders table)	No (stores pointer)
Count per Table	1 only	Many allowed
Storage	Same as table	Extra structure
Speed	Fastest for range/key lookups	Fast for specific column searches
Example	Primary Key (EmpID)	EmpName, Job, DeptID

Quick Real-Life Analogy

Scenario	Explanation
Clustered Index	A phonebook arranged A–Z — data itself sorted.
Non-Clustered Index	The index at the back of a textbook — points you to the page number.

Sequences & Identity Columns in SQL

Purpose

Both **Sequences** and **Identity Columns** are used to **generate unique numbers automatically**, often for *Primary Keys* (like Employee IDs, Order Numbers, etc.).
They help you avoid manually inserting unique IDs and prevent duplicates.

The Problem Without Auto Numbering

Before — Manual Entry (Error-prone)

EmpID	EmpName	Department
101	John	HR
102	Mary	Finance
102	Raj	IT

Problem:

- Duplicate EmpID (102) entered by mistake
- When 100s of employees are inserted manually, duplicates or missing IDs are common.
- If someone forgets to add the next ID, it breaks relationships between tables (Foreign Keys).

After — Auto Number (Fixed)

Let SQL automatically generate unique numbers for you!

There are two main ways to do it:

1. Sequence Object (used in Oracle, PostgreSQL, SQL Server 2012+)
2. Identity Column (used in SQL Server, MySQL, PostgreSQL)

Sequences (Standalone Object)

What is a Sequence?

A sequence is an independent object that generates incrementing numbers.
You can use it anywhere — across multiple tables if needed.

Example (Oracle / PostgreSQL style)

```
CREATE SEQUENCE Emp_Seq
START WITH 1
INCREMENT BY 1;
```

This creates a sequence like:

Call	Value Returned
NEXTVAL	1
NEXTVAL	2
NEXTVAL	3

Use Sequence While Inserting

```
INSERT INTO Employee (EmpID, EmpName, Department)
VALUES (Emp_Seq.NEXTVAL, 'John', 'HR');
```

After multiple inserts:

EmpID	EmpName	Department
1	John	HR
2	Mary	Finance
3	Raj	IT

SQL automatically picks next number.
No duplicates, no manual tracking.

Real-World Example

Think of a sequence like a **ticket dispenser machine** at a service counter:
Every time you press the button, it gives you the next number (1, 2, 3, ...).

Sequence Settings

You can customize:

```
CREATE SEQUENCE Order_Seq
START WITH 1000      -- First number
INCREMENT BY 10      -- Step value
MAXVALUE 9999        -- Stop at 9999
CYCLE;               -- Start over when max reached
```

Identity Columns (Auto Number Inside Table)

What is an Identity Column?

An **identity column** is built into the table definition —
it auto-generates numbers whenever you insert a new row.

Example (SQL Server / MySQL Style)

```
CREATE TABLE Employee (
  EmpID INT IDENTITY(1,1),   -- Start at 1, increment by 1
  EmpName VARCHAR(50),
  Department VARCHAR(30)
);
```

Now you can insert without specifying EmpID:

```
INSERT INTO Employee (EmpName, Department)
VALUES ('John', 'HR'),
      ('Mary', 'Finance'),
      ('Raj', 'IT');
```

EmpID	EmpName	Department
1	John	HR
2	Mary	Finance
3	Raj	IT

IDs auto-generated!
No duplicate or missing IDs.

Difference Between Sequence vs Identity

Feature	Sequence	Identity Column
Definition	Separate object	Part of table definition
Reuse	Can be shared by multiple tables	Belongs to one table only
Control	You can get NEXTVAL anytime	Generates only during INSERT
Syntax	Emp_Seq.NEXTVAL	IDENTITY(1,1)
Flexibility	More configurable	Simpler for one table use

Problem vs Fix Summary

Problem	Solution	Description
Duplicate IDs	Use Sequence or Identity	SQL auto-generates unique IDs
Manual tracking of IDs	Auto increment	Next value handled by DB
Errors while inserting	DB controls sequence	Prevents missing or repeating IDs
Needing custom patterns	Sequence object	Define start, increment, max, cycle

Real-World Analogy

Concept	Analogy
Sequence	A separate token machine that issues numbers for multiple counters.
Identity Column	Each counter having its own built-in number printer.

Performance & Good Practices

Use **Identity columns** when each table needs its own simple auto ID.
Use **Sequences** when multiple tables share numbering (e.g., Orders, Invoices, Receipts).
Always make the ID **Primary Key** to ensure uniqueness.
Don't reset sequences frequently — they're meant to keep history consistent.

Quick Recap

"When you don't want to type IDs manually —
let the database do it for you.

Sequences are like ticket machines — they hand out the next number.
Identity Columns are like built-in number generators inside your table."

Common Table Expressions (CTEs)

What is a CTE?

A **Common Table Expression (CTE)** is a **temporary result set** — a *named subquery* — that you can **reuse** inside your main SQL query.

Think of it as a "temporary view" created inside your query — only for that query.

The Problem Without a CTE

Let's say you have a table:

EmpID	EmpName	DeptID	Salary
1	John	10	8000
2	Mary	20	6000
3	Raj	10	5000
4	Anita	30	9000
5	Sam	10	4000

Business Requirement

Find employees who earn **above the average salary** in their department.

Without a CTE, you must repeat subqueries:

```
SELECT EmpName, DeptID, Salary
FROM Employee e
WHERE Salary >
    (SELECT AVG(Salary) FROM Employee WHERE DeptID = e.DeptID);
```

It works,

but the inner subquery runs again and again for every employee row — making it slower and messy.

Syntax

```
WITH cte_name (optional_columns) AS (
   SELECT query_here
)
SELECT ...
FROM cte_name
WHERE ...;
```

You can use **multiple CTEs** separated by commas:

```
WITH cte1 AS (...),
    cte2 AS (...)
SELECT ...
FROM cte1 JOIN cte2 ...;
```

The Fix — Using a CTE

We can **calculate once** and **reuse it**!

```
WITH DeptAvg AS (
   SELECT DeptID, AVG(Salary) AS AvgSal
   FROM Employee
   GROUP BY DeptID
)
SELECT e.EmpName, e.DeptID, e.Salary, d.AvgSal
FROM Employee e
JOIN DeptAvg d ON e.DeptID = d.DeptID
WHERE e.Salary > d.AvgSal;
```

Output

EmpName	DeptID	Salary	AvgSal
John	10	8000	5666.67
Anita	30	9000	9000.00

Explanation

- The CTE (DeptAvg) calculates average salary *once* per department.
- The main query joins Employee with that temporary result.
- Result = cleaner code, faster performance, and easier maintenance.

Real-World Use Case

Let's say your manager asks:

"Give me the top 3 highest-paid employees from each department."

Using CTEs:

```
WITH RankedEmp AS (
  SELECT EmpID, EmpName, DeptID, Salary,
      ROW_NUMBER() OVER (PARTITION BY DeptID ORDER BY Salary DESC) AS RankNo
  FROM Employee
)
SELECT EmpName, DeptID, Salary
FROM RankedEmp
WHERE RankNo <= 3;
```

Easy to read.
No nested mess.
Can be reused in the same query.

Problem vs Fix Summary

Problem	Fix	Explanation
Repeated logic in subqueries	Use a CTE	Write once, use many times
Hard-to-read queries	Use named temporary blocks	Improves readability
Recomputing values	Use CTE to compute once	Improves performance

Real-World Analogy

Concept	Analogy
Subquery	Asking “What’s the total?” every time for each person
CTE	Calculating totals once, writing them on a whiteboard, and then referring to that board for all your answers

Real-World Example: Reporting

In Power BI or analytics pipelines, you’ll often write SQL with CTEs to:

- Calculate aggregates (totals, averages)
- Apply filters in multiple stages
- Combine multiple datasets cleanly

Makes dashboards and reports faster and easier to maintain.

STORED PROCEDURES in SQL (with Parameters, Input/Output, and Error Handling)

What is a Stored Procedure?

A **Stored Procedure** is a **predefined block of SQL code** — saved in the database — that you can execute whenever you need.
It's like a *function in programming* but for SQL operations.

Instead of writing the same query again and again, you save it once and *call* it by name.

The Problem Without Stored Procedures

Let's say your manager asks:

"Find total salary for a department — we'll need it frequently for reports."

Without a stored procedure:
You (and others) must **type or paste** this query every time:

```
SELECT SUM(Salary)

FROM Employee

WHERE DeptID = 10;
```

Later, for DeptID 20, 30, etc., people copy-paste the same query with different IDs.
Too much repetition
Prone to mistakes
No consistency or security

The Fix — Create a Stored Procedure

Save the logic once. Then just call it whenever needed.

Example 1: Simple Stored Procedure

```
CREATE PROCEDURE GetTotalSalary

AS

BEGIN

  SELECT DeptID, SUM(Salary) AS TotalSalary

  FROM Employee

  GROUP BY DeptID;

END;
```

Now, anyone can simply run:

```
EXEC GetTotalSalary;
```

Easy
Consistent
Reusable

Adding Parameters (Input & Output)

Parameters make procedures **dynamic and reusable**.

Input Parameter Example

```
CREATE PROCEDURE GetDeptSalary
  @Dept INT
AS
BEGIN
  SELECT DeptID, SUM(Salary) AS TotalSalary
  FROM Employee
  WHERE DeptID = @Dept
  GROUP BY DeptID;
END;
```

Now run:

```
EXEC GetDeptSalary @Dept = 10;
```

Fetches only Dept 10's total salary.

Explanation

- @Dept = Input parameter → acts like a variable.
- Makes your procedure flexible for any department.

Output Parameter Example

```
CREATE PROCEDURE GetDeptTotal
  @Dept INT,
  @TotalSalary INT OUTPUT
AS
BEGIN
  SELECT @TotalSalary = SUM(Salary)
  FROM Employee
  WHERE DeptID = @Dept;
END;
```

Run like this:

```
DECLARE @Total INT;
EXEC GetDeptTotal @Dept = 10, @TotalSalary = @Total OUTPUT;
PRINT @Total;
```

Now your program (or script) can **capture** the result in a variable.

Adding Error Handling

In real systems, procedures must handle errors gracefully —
for example, when a department doesn't exist.

Error Handling Example

```
CREATE PROCEDURE GetDeptSalarySafe
  @Dept INT
AS
BEGIN
  BEGIN TRY
    IF NOT EXISTS (SELECT 1 FROM Department WHERE DeptID = @Dept)
      THROW 50001, 'Invalid Department ID!', 1;

    SELECT DeptID, SUM(Salary) AS TotalSalary
    FROM Employee
    WHERE DeptID = @Dept
    GROUP BY DeptID;
  END TRY
  BEGIN CATCH
    SELECT ERROR_MESSAGE() AS ErrorMessage;
  END CATCH
END;
```

Run:

```
EXEC GetDeptSalarySafe @Dept = 99;
```

Output:

Invalid Department ID!

Handles errors cleanly.
Prevents your program from crashing.

Real-World Use Cases

Use Case	What the Procedure Does
Payroll System	Calculate monthly salary and bonuses
Banking	Transfer funds securely between accounts
HR	Generate employee reports by department
E-commerce	Update stock after each order
Data Warehouse	Clean and transform data before loading

Why Stored Procedures Are Powerful

Benefit	Description
Reusability	Write once, run many times
Performance	Precompiled by SQL engine (faster)
Security	Hide logic; control access to data
Modularity	Build reusable building blocks
Error Handling	Prevents crashes and invalid updates

User-Defined Functions (UDF) vs Stored Functions – Comparison Table

Feature	User-Defined Function (UDF)	Stored Function
Definition	**A custom function written by the user to reuse logic in SQL (mainly in analytics engines).**	**A function stored permanently inside a relational database and returns a single value.**
Where used?	**Analytics platforms (Databricks, Spark, Hive, BigQuery, Snowflake).**	**OLTP/Relational DBs (MySQL, PostgreSQL, SQL Server, Oracle).**

Feature	User-Defined Function (UDF)	Stored Function
Purpose	Data transformation, calculations, cleaning.	Enforcing business rules and returning one specific value.
Output	Can return a value or even a table (depending on system).	Always returns one value.
Performance	Can be slower (especially Python UDFs) because execution happens row-by-row.	Usually faster and optimized because DB engine compiles & caches them.
Who uses it?	Data Engineers, Data Analysts, BI Developers.	Backend Developers, DBAs, System Architects.
Stored in DB?	Not always stored inside the database; depends on platform.	Always stored in the database like a permanent object.
Can modify data (INSERT/UPDATE)?	❌ No.	❌ No (stored procedures can, but stored functions cannot).
Reusability	High, reused in queries across notebooks/jobs.	High, reused across SQL programs and applications.
Best Use Case	Repeating transformations in analytic workflows.	Consistent business logic in transactional databases.